Artificial Intelligence 310 (2022) 103752

Contents lists available at ScienceDirect

Artificial Intelligence

www.elsevier.com/locate/artint

Priority inheritance with backtracking for iterative multi-agent path finding

Keisuke Okumura [a,*], Manao Machida [b], Xavier Défago [a], Yasumasa Tamura [a]

[a] School of Computing, Tokyo Institute of Technology, Tokyo, Japan
[b] NEC Corporation, Tokyo, Japan

ARTICLE INFO

Article history:
Received 22 November 2021
Received in revised form 20 June 2022
Accepted 26 June 2022
Available online 3 July 2022

Keywords:
Multi-agent pathfinding
Online and lifelong planning
Multi-robot coordination

ABSTRACT

In the Multi-Agent Path Finding (MAPF) problem, a set of agents moving on a graph must reach their own respective destinations without inter-agent collisions. In practical MAPF applications such as navigation in automated warehouses, where occasionally there are hundreds or more agents, MAPF must be solved iteratively online on a lifelong basis. Such scenarios rule out simple adaptations of offline compute-intensive optimal approaches; and scalable sub-optimal algorithms are hence appealing for such settings. Ideal algorithms are scalable, applicable to iterative scenarios, and output plausible solutions in predictable computation time.

For the aforementioned purpose, this study presents Priority Inheritance with Backtracking (PIBT), a novel sub-optimal algorithm to solve MAPF iteratively. PIBT relies on an adaptive prioritization scheme to focus on the adjacent movements of multiple agents; hence it can be applied to several domains. We prove that, regardless of their number, all agents are guaranteed to reach their destination within finite time when the environment is a graph such that all pairs of adjacent nodes belong to a simple cycle (e.g., biconnected). Experimental results covering various scenarios, including a demonstration with real robots, reveal the benefits of the proposed method. Even with hundreds of agents, PIBT yields acceptable solutions almost immediately and can solve large instances that other established MAPF methods cannot. In addition, PIBT outperforms an existing approach on an iterative scenario of conveying packages in an automated warehouse in both runtime and solution quality.

1. Introduction

In systems with multiple moving agents, valid paths must be provided to ensure that agents reach their destinations smoothly without any collisions, while minimizing excess travel time. This problem is embodied in *Multi-Agent Path Finding (MAPF)* [1], which aims to find a set of collision-free paths on graphs. Currently, MAPF is receiving considerable attention because it is highly applicable in problems such as intersection management [2], automated warehouse [3], games [4], airport surface operation [5], and parking [6]. However, the optimization of this problem is known to be computationally intractable [7–10] because of the exponential growth of the search space as the number of agents increases.

* Corresponding author.
E-mail addresses: okumura.k@coord.c.titech.ac.jp (K. Okumura), manaomachida@nec.com (M. Machida), defago@c.titech.ac.jp (X. Défago), tamura@c.titech.ac.jp (Y. Tamura).

https://doi.org/10.1016/j.artint.2022.103752

Prior research on MAPF primarily focuses on solving a "one-shot" version of the problem, where agents reach their respective goals from their initial positions only once. In practical applications such as conveying packages in a warehouse with hundreds or more robots [3], MAPF must be solved *online* and *iteratively* on a *lifelong* basis. That is, whenever an agent reaches a goal, it receives a new one, and the plan, a set of paths, is updated in real-time. Such scenarios rule out simple adaptations of offline and compute-intensive optimal approaches because, despite state-of-the-art optimal algorithms [11, 12], it is challenging to find solutions for a few hundred agents within a realistic timeframe. For this purpose, an attractive possibility is to design scalable sub-optimal algorithms that output plausible solutions within a predictable computational time, which also works in iterative situations. Such algorithms are also promising for real-time planning where deliberation time is limited because it enables iterative refinement for known MAPF solutions [13].

To this end, a novel sub-optimal algorithm named *Priority Inheritance with Backtracking (PIBT)* is proposed to solve MAPF iteratively. Theoretically, PIBT ensures *reachability*, that is, every agent always reaches its destination within a finite time when the environment is a graph such that all pairs of adjacent nodes belong to a simple cycle (e.g., biconnected). PIBT also has low-cost time complexity. It repeats one-timestep planning until termination (e.g., until all agents have reached their destinations). For each timestep, PIBT requires the time complexity $O(|A| \cdot (\Delta(G) + F + \log |A|))$, where $|A|$ is the number of agents, $\Delta(G)$ is a diameter of a graph G, and F is the time required to evaluate the distance from one vertex to the destination. Note that F can be constant with preprocessing. As a result, PIBT is expected to work in a short runtime, even in the case of massive problems.

From the empirical end, PIBT immediately returns acceptable solutions despite hundreds or more agents in running times that are orders of magnitudes faster than other established MAPF approaches, such as sub-optimal priority-based [4], rule-based [14], and search-based [15] algorithms, as well as state-of-the-art optimal search-based [16] and compiling-based [17] algorithms. For instance, using an ordinary laptop, we observe that PIBT solves one-shot MAPF in a large grid map (530×481, number of vertices: $43,151$) from [1] with $1,000$ agents in at most 5 seconds, while keeping sub-optimality below 1.5 on average. Furthermore, PIBT outperforms an existing approach for both runtime and solution quality on an iterative scenario of conveying packages in an automated warehouse, called Multi-Agent Pickup and Delivery (MAPD) [18].

Reachability does not ensure that all agents are on their goals simultaneously; hence, PIBT is incomplete for conventional MAPF. However, agents often do not need to stay at their goals in real-life applications such as lifelong delivery tasks. Reachability is convenient in such scenarios. Indeed, this study introduces the application of PIBT to a lifelong variant of MAPF (i.e., MAPD [18]) while ensuring completeness.

The mechanism of PIBT is simple and easy to implement. The algorithm focuses on the adjacent movements of multiple agents based on prioritized planning [19] in a unit-length time window. Priority inheritance is a well-known approach for dealing effectively with priority inversion in real-time systems [20] and is applied here to path adjustment. When a low-priority agent–X impedes the movement of a higher-priority agent–Y, agent–X temporarily inherits the higher-priority of agent–Y. To avoid agents getting stuck waiting, priority inheritance is executed in combination with a backtracking protocol. Because this mechanism only requires local interactions between agents, PIBT has a high potential for decentralized implementations.

In summary, the main contributions are two-fold:

1. We propose the PIBT algorithm, which solves MAPF iteratively and guarantees reachability.
2. We evaluate the algorithm in various environments to assess its practicality. In particular, experimental results analyzing various scenarios, including robot demonstration, confirm the adequacy of the algorithm both in finding paths in large environments with many agents as well as in conveying packages in an automated warehouse.

All external materials (e.g., code and a movie) are available at https://kei18.github.io/pibt2. The techniques presented here provide practical insight into online and lifelong scenarios involving a swarm of agents.

The rest of the paper is organized as follows. Section 2 reviews existing MAPF studies. Section 3 defines MAPF and its variant called MAPD, a problem considering operations in automated warehouses. Then, Section 4 presents the PIBT algorithm along with a theoretical analysis of its reachability and time complexity. In Section 5, we empirically evaluate PIBT using many simulated scenarios and a demonstration on real robots. Finally, Section 6 concludes the paper and discusses future directions.

Difference from conference versions The PIBT algorithm was initially presented in our preliminary paper [21]. PIBT$^+$ (Sec. 4.4.1) was initially mentioned in [13]; however, theoretical analyses were not provided; we remedy this shortfall here. This paper, in addition to improving the description and presentation of PIBT, has the following major differences from the previous versions: (1) Exhaustive experiments were conducted on one-shot MAPF using various fields and solvers. As a result, this paper thoroughly clarifies the advantages and disadvantages of PIBT compared to other de facto approaches. The implementation was also significantly improved, with the running time decreased by orders-of-magnitudes than that of the conference version [21]. This is mainly due to distance evaluation, as explained in Section 4.4.1. (2) Further theoretical analyses were conducted on PIBT, for instance, on reachability considering rotation conflicts. (3) A stress test for the number of agents and the robot demonstration of PIBT in an iterative scenario were added.

2. Related work

This section describes the basis of MAPF, summarizes the relationship between PIBT and other existing MAPF algorithms, and presents lifelong and online situations apart from conventional MAPF studies.

2.1. Basis of multi-agent path finding (MAPF)

The *MAPF* [1] assigns each agent on a graph a collision-free path to its destination. Time is discrete; for each timestep, each agent performs atomic action in a synchronized manner such that it either moves to an adjacent node or stays in its current location. MAPF is known to be an NP-hard problem according to various optimization criteria [22,7], which holds for restricting fields in planar graphs [9], in grid structures [10], and in approximating within any constant factor less than 4/3 for the last arrival time metric [8]. Limiting the discussion to sub-optimal solutions on undirected graphs, there exists a procedure of $O(|V|^3)$ operations [23] for the compatibility of the pebble motion problem (a generalization of the sliding tile puzzle), where $|V|$ is the number of vertices. In contrast, finding solutions on directed graphs itself is NP-complete [24]. The case of strongly biconnected digraphs is special, where there is a polynomial-time procedure [25] to solve MAPF for such graphs. *The proposed method, PIBT, is neither complete nor optimal for MAPF. However, PIBT works with both directed and undirected graphs.*

2.2. Algorithms

To date, the numerous solvers developed for MAPF can be categorized into optimal or sub-optimal, complete or incomplete. Here, an algorithm is *complete* when it ensures to return a set of collision-free paths such that the end of each path is the destination of each agent. If no such set of paths exists, complete algorithms must report its absence. Another categorization is solving style, by which algorithms are classified into the following five classes:

- *Search-based* approaches search feasible solutions in a coupled manner among agents. Examples are A* with operator decomposition [26], increasing cost tree search [27], enhanced partial expansion A* [28], conflict-based search (CBS) [16], and M* [29]. These algorithms are complete and can solve MAPF optimally. Bounded sub-optimal versions [30,15], which are complete, have also been developed to solve large instances.
- *Compiling-based* approaches reduce MAPF to well-known problems, such as, SAT [31–33], integer linear programming [34,17], and answer set programming [35,36]. Those algorithms are complete and optimal.
- ***Prioritized planning*** [19] approaches sequentially plan individual paths for agents in a decoupled manner; in general, these algorithms are neither complete nor optimal.
- ***Rule-based*** approaches make agents move step-by-step following ad-hoc rules. Examples include a graph abstraction approach [37], [38] for spanning trees, BIBOX [39] for biconnected graphs, and push and swap/rotate [14,40] for arbitrary graphs. The rule-based approaches are sub-optimal but often ensure completeness for certain problem instances.
- *Learning-based* approaches use machine learning techniques and imitate good behavior of agents for expert data. Examples include reinforcement learning [41,42] and graph neural networks [43,44]. They are suboptimal and incomplete.

See [45,46] for comprehensive reviews. Despite state-of-the-art search-based or compiling-based optimal approaches, planning with a few hundred agents within a reasonable time is challenging [12,11]. Learning-based approaches scale well in certain cases; however, they do not have any theoretical guarantees. Therefore, for the target scenarios considered in this study (i.e., online and lifelong scenarios with hundreds or more agents), prioritized planning and rule-based approaches are promising methods. Indeed, PIBT combines these two classes. The following reviews focus on prioritized planning and rule-based approaches. Note that PIBT is "incomplete" because it ensures that all agents eventually reach their destinations; however, they may not be there simultaneously. This property is designed for lifelong situations and significantly differs from the theoretical properties provided by existing algorithms for one-shot scenarios.

Prioritized planning [19] is neither complete nor optimal; however, it is a computationally inexpensive approach to MAPF. It is based on the concept that agents sequentially plan paths according to their unique priorities while avoiding conflicts with previously planned paths. Because of its computational efficiency and simplicity, prioritized planning is highly favorable in practice and is often used as part of MAPF solvers. For instance, Wang and Botea [47] combined techniques of prioritized planning for the way the blanks moved around in sliding tile puzzles and proposed a complete algorithm for a specific class of instances. Velagapudi *et al.* [48] proposed decentralized implementations of prioritized planning that divide planning into several iterations. Variants of prioritized planning have also been studied, e.g., multi-robot decoupled planning that first computes individual plans neglecting collisions, and then replans individual paths while incorporating collision costs gradually [49]. We acknowledge that this scheme has similarities to PIBT, albeit the applied problem is different and the approach has no theoretical guarantee. The well-known algorithm of prioritized planning for MAPF is hierarchical cooperative A* (HCA*) [4]. The "windowed" version of HCA* is known as windowed HCA* (WHCA*) [4], which repeatedly plans fixed-length (windowed) paths for all agents. PIBT can be considered as using WHCA* with a unit-length window sizes.

Because priority ordering is crucial for prioritized planning, several studies addressed this issue. The negotiation process between agents regarding the ordering was studied in [50]; this process solves conflicts by having involved agents try all priority orderings and deals with congestion by limiting negotiation to at most three agents while letting others wait. A similar approach is taken in [51] where the winner determines the prioritization scheme. There are some heuristic approaches for offline planning to find a reasonable ordering, for instance, adjusting the ordering by simple hill-climbing [52], or using distances between initial locations and destinations [53]. There is a sufficient condition that the sequential collision-free solution can always be constructed regardless of priority orders, called well-formed instances [54], such that for each pair of start and goal, a path exists that traverses no other starts and goals. However, well-formed instances are difficult to realize in dense situations. A recent theoretical analysis for prioritized planning [55] identifies instances that fail for any order of *static* priorities, which provides a strong case for planning based on *dynamic* priorities, such as the approach taken with PIBT. The paper also presents a variant of prioritized planning that searches for good static priorities using techniques from CBS; however, the method has no theoretical guarantee.

Push and swap/rotate (PS) [14,40], which partly influenced the proposed PIBT, is a sub-optimal rule-based approach for arbitrary graphs. It relies on two primitives: the "push" operation to move an agent toward its goal and the "swap" operation to allow two agents to swap locations without altering the configuration of other agents. These approaches, however, only allow a single agent or a pair of agents to move at a time. Some studies enhanced PS, for instance, parallel push and swap [56] by enabling all agents to move simultaneously, or push–swap–wait [57] by taking a decentralized approach in narrow passages. Wei *et al.* [58] proposed a decentralized approach that partly uses the push and swap technique to avoid deadlocks. DisCoF [59], a decentralized method for MAPF, also uses swap operation to ensure completeness. There is an algorithm similar to PS, called tree-based agent swapping strategy (TASS) [60], targeting trees. PIBT can be regarded as a combination of safe "push" operations because of the backtracking protocol and dynamic priorities. Note that PIBT does not require the operational equivalent of "swap."

2.3. Lifelong and online situations

Conventional MAPF focuses on solving a *"one-shot"*[1] and *offline* version of the problem, that is, ensuring that agents reach their goal from their initial position just once. Several variants of the MAPF problem [18,61,62] aim to address the needs of more realistic scenarios, including lifelong and online operations. Among them, *Multi-Agent Pickup and Delivery (MAPD)* [18] abstracts real scenarios such as an automated warehouse and has both task allocation and path planning. Agents are assigned to a task from a stream of delivery tasks and must consecutively visit both a pickup and a delivery location. The study proposes two decoupled algorithms based on HCA* for MAPD, called token passing and token passing with task swap, respectively. These algorithms can be decentralized but require a certain amount of non-task endpoints where agents do not block other agents' motions. *Online MAPF* [61] addresses situations with a dynamic group of agents, i.e., new agents might appear at runtime, or agents might disappear when they reach their destinations. We do not address this problem; however, the adaptation of PIBT for this is straightforward because PIBT essentially repeats one-timestep planning.

3. Problem definition

This study focuses on two problems: conventional one-shot MAPF problem [1] and one of its variants, the MAPD problem [18] for lifelong and online scenarios. We formulate both problems in this section. Note that the adaptation of PIBT to other lifelong MAPF variants [61,62] is straightforward.

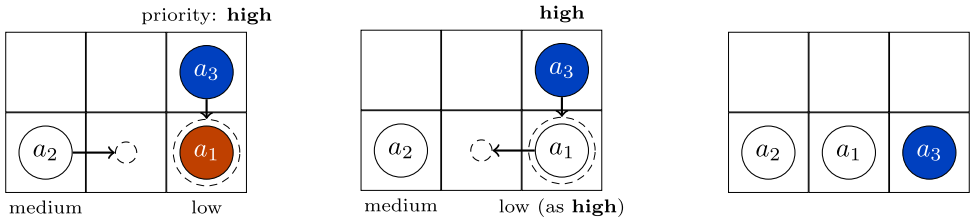
3.1. Multi-agent path finding (MAPF)

An instance of *MAPF* is given by a directed graph $G = (V, E)$, a set of agents $A = \{a_1, \dots, a_n\}$ with $|A| \leq |V|$, an injective initial state function $s : A \mapsto V$, and an injective goal state function $g : A \mapsto V$. To simplify the notation, let s_i and g_i denote $s(a_i)$ and $g(a_i)$, respectively. Considering practical situations, a graph G must satisfy the following two properties: (1) *simple*, i.e., neither (self) loops nor multiple edges (between the same vertices) are present. (2) *strongly-connected*, i.e., every node is reachable from every other node. This includes all simple undirected graphs that are connected.

Let $\pi_i[t] \in V$ denote the location of an agent a_i at discrete time $t \in \mathbb{N}$. At each timestep t, a_i can move to an adjacent node, or stay at its current location, i.e., $\pi_i[t+1] \in Neigh(\pi_i[t]) \cup \{\pi_i[t]\}$, where $Neigh(v)$ is the set of nodes neighboring node v in graph G. Agents must avoid two types of conflicts: (1) *vertex conflict*: $\exists i, j,\ i \neq j,\ \pi_i[t] = \pi_j[t]$ (when agents share the same location), and, (2) *swap conflict*: $\exists i, j,\ i \neq j,\ \pi_i[t] = \pi_j[t+1] \wedge \pi_i[t+1] = \pi_j[t]$ (when two agents swap their occupying vertices). We refer to a set of paths as *conflict-free* when those two conditions are satisfied.

A solution to *MAPF* is a set of conflict-free paths $\{\pi_1, \dots, \pi_n\}$ such that all agents reach their goals at a certain timestep T. More precisely, the problem assigns a path $\pi_i = (\pi_i[0], \pi_i[1], \dots, \pi_i[T])$ to each agent such that $\pi_i[0] = s_i$ and $\pi_i[T] = g_i$. Note that all agents must be at their goals at the same timestep. Hence, T is common among agents.

[1] A *"one-shot"* problem is a problem that has clearly defined inputs and must be solved only once. This is in contrast to problems that must be solved repeatedly for a *stream* of inputs.



(a) Stuck agent (b) Priority inheritance (c) One timestep later

Fig. 1. Examples of priority inheritance. Circles with solid lines represent agents. Requests for the next timestep are depicted by dashed circles, determined greedily according to agents' destinations (omitted here). Without inheritance (1a), a stuck agent (a_1) cannot give way to a high-priority agent (a_3) without risking collision into a third agent (a_2). With priority inheritance (1b), a_1 temporarily inherits the priority of a_3 forcing a_2 to solve the situation (1c).

Two different metrics are commonly used to evaluate the quality of MAPF solutions: (1) *sum-of-costs*: $\sum_{a_i \in A} T_i$ where T_i is the earliest timestep such that $\pi_i[T_i] = g_i, \ldots, \pi_i[T] = g_i, T_i \leq T$, (2) *makespan*, i.e., the value of T.

3.2. Multi-agent pickup and delivery (MAPD)

Similar to MAPF, a graph $G = (V, E)$ and a set of agents $A = \{a_1, \ldots, a_n\}$ with $|A| \leq |V|$, and an injective initial state function $s : A \mapsto V$ (for simplicity, $s(a_i)$ is denoted as s_i), are given.

Consider a stream of tasks $\Gamma = \{\tau_1, \tau_2, \ldots\}$. A task τ_j is defined as a tuple $(v_{pickup}, v_{delivery})$, where $v_{pickup}, v_{delivery} \in V$. MAPD includes situations in which tasks are added to Γ as time progresses. In other words, it is not assumed that all tasks are known initially. An agent is *free* when it has no assigned task. A task $\tau_j \in \Gamma$ can only be assigned to free agents and is assigned to at most one agent. When τ_j is assigned to a_i, a_i has to visit v_{pickup} and $v_{delivery}$ in order. When the two nodes have been visited by a_i, τ_j is completed, and a_i becomes free again. A *service time* of τ_j is defined as the time interval from the generation of τ_j to its completion. Similar to MAPF, time is discretized. At each timestep, each agent can move to an adjacent node or stay at its current node, provided that no two agents collide, i.e., vertex and swap conflicts are prohibited.

The objective is to complete all tasks as quickly as possible. A solution to *MAPD* is a set of conflict-free paths and task assignments, with each agent starting from s_i. When $|\Gamma|$ is finite, an MAPD algorithm is *complete* as it ensures that all tasks are finished within a finite number of timesteps.

Two kinds of objective functions are considered: (1) *service time* and (2) *makespan*, that is, the timestep when all tasks are completed. Note that makespan is only defined when Γ is finite.

3.3. Other notations

The diameter of graph G is denoted by $diam(G)$, and its maximum degree by $\Delta(G)$. Let $dist(u, v)$ denote the shortest path length from $u \in V$ to $v \in V$.

4. Priority inheritance with backtracking (PIBT)

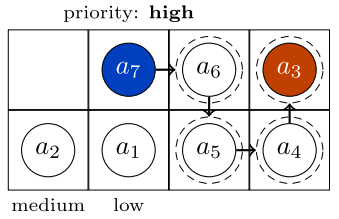
This section introduces the PIBT algorithm. The concept of PIBT is initially explained by intuitive examples. To avoid unnecessarily complex explanations, PIBT is introduced as a centralized algorithm, focusing on analyzing the prioritization scheme itself. Note that PIBT relies on a decoupled approach, rendering it readily amenable to decentralization, as briefly discussed later. After providing a theoretical analysis on PIBT including reachability, how to apply PIBT to specific problems, such as one-shot MAPF and MAPD, is described.

4.1. Concept

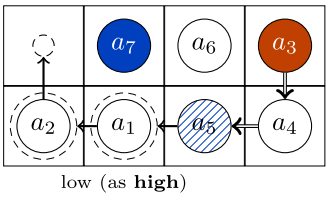
Aiming to solve MAPF iteratively, PIBT repeats one-timestep prioritized planning until it is terminated. At every timestep, each agent first updates its unique priority. Then, the agents sequentially determine their next location in decreasing order of priorities while avoiding nodes that have been requested by higher-priority agents. Prioritization alone, however, can still cause deadlocks. As shown in Fig. 1a, a stuck agent a_1 cannot go anywhere without collisions with other agents; hence, this situation can be regarded as a certain kind of deadlock.

4.1.1. Priority inheritance

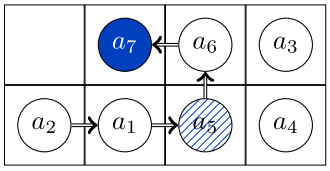
A situation with a stuck agent can also be regarded as a case of *priority inversion*, in the sense that a low-priority agent (a_1) holding a resource claimed by a higher-priority agent (a_3) fails to obtain a second resource held by an agent with medium priority a_2. A classical way to deal with priority inversion is to rely on *priority inheritance* [20] (Fig. 1b). The rationale is that a low-priority agent (a_1) temporarily inherits the higher priority of the agent (a_3) claiming the resources it holds, thus forcing medium-priority agents (a_2) out of the way (Fig. 1c).



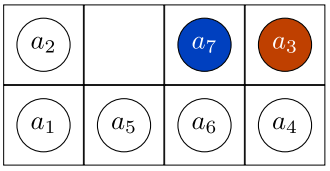
(a) Priority inheritance



(b) Backtracking and priority inheritance again



(c) Backtracking



(d) one timestep later

Fig. 2. Example of PIBT. The flow of back tracking is indicated by double-lined arrows. Because a_3 is stuck (2a), backtracking returns as invalid to a_4, and subsequently to a_5. a_5 executes other priority inheritance to a_1 (2b). a_1, a_5, a_6 and a_7 wait for the results of back tracking (2c) and then start moving (2d).

4.1.2. Backtracking

Priority inheritance deals effectively with priority inversion; however, it does not completely ensure deadlock freedom. For instance, as shown in Fig. 2a, an agent a_3 cannot escape as a result of consecutive priority inheritances ($a_7 \to a_6 \to a_5 \to a_4$).

The solution relies on *backtracking*; any agent a_i that executes priority inheritance must wait for an outcome, valid or invalid. If valid, a_i successfully moves to the desired node. Otherwise, it must request a different node, excluding: (1) nodes requested by a higher priority agent, and (2) nodes having already returned an invalid outcome. Upon finding no valid or unoccupied nodes, a_i sends back an invalid outcome to the agent from which it inherited its priority.

For instance, in Fig. 2b, a_3 sends "invalid" back to a_4 as the outcome of the request received previously from a_4 (Fig. 2a). a_4 tries to replan its next location; however, in the absence of other alternatives, in turn sends invalid to a_5. Because a_1 has lower priority, a_5 can let a_1 inherit its priority as an alternative, which leads to a valid outcome (Fig. 2c), and agents can move (Fig. 2d).

By combining priority inheritance and backtracking, it is ensured that all agents are assigned to the next locations without collisions.

4.2. Algorithm

We formalize the concept in Algorithm 1, which consists of (1) a top-level procedure [Lines 3–7] calling (2) a recursive procedure [Lines 8–22] that executes priority inheritance and backtracking.

Top-level procedure PIBT first updates the priority $p_i \in \mathbb{R}_{\geq 0}$ for each agent a_i [Line 3]. The rule is simple; increment p_i when a_i has not reached its goal g_i; otherwise, reset p_i to $\epsilon_i \in [0, 1)$. The tie-breaker ϵ_i, which is distinct for agents, has the role of keeping p_i unique among agents. Subsequently, according to priorities, the algorithm assigns locations to each agent a_i without a next location $\pi_i[t+1]$ via the procedure PIBT [Line 6]. The termination of the algorithm is explained later.

Recursive procedure The procedure PIBT implements the concept of priority inheritance and backtracking. It has two arguments: the agent a_i making a decision and an agent a_j from which a_i inherits its priority, or $\perp$ if there is none. The procedure returns `invalid` if a_i becomes a stuck agent like the red agent a_3 in Fig. 2, otherwise returns `valid`. Once this procedure is called, a_i must determine $\pi_i[t+1]$ from the ordered set of candidate nodes C. Here, C consists of $\pi_i[t]$ and its neighbors [Line 9], sorted in increasing order of distance from the goal g_i [Line 10]. To avoid unnecessary priority inheritance, the presence (or absence) of an agent is used as a tie-breaking rule. C is filtered, excluding two kinds of nodes: (1) nodes already requested by someone (to avoid vertex conflict) [Line 12] and (2) $\pi_j[t]$ if $a_j \neq \perp$ (to avoid swap conflict) [Line 13]. If no nodes remain in C, this is the case of a stuck agent; then, a_i must stay at the current location and the procedure returns `invalid` [Lines 20–21].

For each node $v \in C$, the procedure checks whether a_i can move to v in the next timestep. After reserving v for a_i [Line 14], the priority inheritance occurs from a_i to a_k when another agent a_k, which has not yet determined the next

Algorithm 1 PIBT.

Input: graph G, agents A, starts $\{s_1, \dots s_n\}$, goals $\{g_1, \dots g_n\}$
Output: paths $\{\pi_1, \dots, \pi_n\}$
Preface: $\pi_i[t]$ is assumed to be initialized with $\perp$
1: $\pi_i[0] \leftarrow s_i$: for each agent $a_i \in A$
2: $p_i \leftarrow \epsilon_i$: for each agent $a_i \in A$ s.t. $\epsilon_i \in [0, 1)$ and $\epsilon_i \neq \epsilon_j (i \neq j)$ ▷ setup priorities

(for each timestep $t = 1, 2, \dots$ until terminates, repeat the following)
3: $p_i \leftarrow$ **if** $\pi_i[t] = g_i$ **then** ϵ_i **else** $p_i + 1$: for each agent $a_i \in A$ ▷ update priorities
4: sort A in decreasing order of priorities p_i
5: **for** $a_i \in A$ **do**
6: **if** $\pi_i[t+1] = \perp$ **then** PIBT$(a_i, \perp)$
7: **end for**

8: **procedure** PIBT(a_i, a_j) ▷ we use t implicitly
9: $C \leftarrow Neigh(\pi_i[t]) \cup \{\pi_i[t]\}$
10: sort C in increasing order of $dist(u, g_i)$ where $u \in C$ ▷ tie-break: presence of agents
11: **for** $v \in C$ **do**
12: **if** $\exists a_k \in A$ s.t. $\pi_k[t+1] = v$ **then continue** ▷ avoid vertex conflict
13: **if** $a_j \neq \perp \wedge \pi_j[t] = v$ **then continue** ▷ avoid swap conflict
14: $\pi_i[t+1] \leftarrow v$ ▷ reserve
15: **if** $\exists a_k \in A$ s.t. $\pi_k[t] = v \wedge \pi_k[t+1] = \perp$ **then**
16: **if** PIBT(a_k, a_i) is `invalid` **then continue** ▷ replanning
17: **end if**
18: **return** `valid`
19: **end for**
20: $\pi_i[t+1] \leftarrow \pi_i[t]$
21: **return** `invalid`
22: **end procedure**

location, occupies v [Lines 15–17]. If a_k returns `invalid`, the next location for a_i is replanned [Line 16], or else, a_i finishes the planning and returns `valid`, as well as that v is unoccupied by others [Line 18].

A step-by-step example of Algorithm 1 is illustrated in Fig. 3.

4.3. Theoretical analysis

4.3.1. Reachability

Here, it is proven that all agents eventually reach their respective destination in graphs with adequate properties (e.g., biconnected graphs). The proof relies on the following Lemma 1, which states that the agent with the highest priority at each timestep is always assigned to its desired node, assuming that it can always move along a simple cycle.

Lemma 1. *Let a_1 denote the agent with highest priority at timestep t and v^* the nearest node to g_1 among neighbors of $\pi_1[t]$. If a simple cycle $\boldsymbol{C} = (\pi_1[t], v^*, \dots)$ exists, Algorithm 1 assigns v^* to a_1 as $\pi_1[t+1]$.*

Proof. Fig. 4 visualizes the following.

Assume that the top-level procedure calls PIBT$(a_1, \perp)$ at Line 6 for the agent with highest priority. At this phase, $\pi_i[t+1]$ is $\perp$ for any agent a_i, i.e., the next location has not been assigned yet. Consequently, a_1 selects v^* as a target node v in the first iteration of Lines 5–7. Then, it remains to prove that Line 16, priority inheritance, never returns `invalid` when another agent a_2 occupies v^*. In the other cases, a_1 evident gets v^*.

Next, assume that PIBT(a_2, a_1) is called from the original PIBT$(a_1, \perp)$. The existence of the cycle $\boldsymbol{C} = (\pi_1[t], v^*)$ guarantees that a_2 has a non-empty set of candidate nodes C_2 for the next timestep without colliding with a_1. During the iterations of Lines 11–19, PIBT(a_2, a_1) returns `valid` when an unoccupied node $v \in C_2$ at timestep t is selected. This forms the basis of the remaining proof by induction.

Following this, suppose that PIBT(a_i, a_{i-1}) is called before PIBT(a_2, a_1) returns any value. Similar to the case of a_2, PIBT(a_i, a_{i-1}) must return `valid` when one of the surrounding nodes of $\pi_i[t]$ is unoccupied at timestep t. In addition, as distinct from a_2, which must avoid swap conflict with a_1, a_i can select $\pi_1[t]$ and return `valid` when $\pi_1[t]$ is neighbor to $\pi_i[t]$. As a result, PIBT(a_{i-1}, a_{i-2}), which calls PIBT(a_i, a_{i-1}) also returns `valid` and eventually, PIBT(a_2, a_1) returns `valid`.

Next, by contradiction, suppose that PIBT(a_2, a_1) returns `invalid`. This assumption means that, for all agents $a_j (\neq a_1)$ neighboring a_2 at timestep t, PIBT$(a_j, \cdot)$ returns `invalid`. This is the same for a_j, and so forth. Meanwhile, the existence of $\boldsymbol{C}$ indicates that at least one agent $a_k \neq a_2$ such that $\pi_k[t] \in \boldsymbol{C}$ had initially at least one free neighbor node. Even though all nodes in $\boldsymbol{C}$ are occupied, the agent on the last node of $\boldsymbol{C}$ has a free neighbor node, i.e., $\pi_1[t]$. This is a contradiction; PIBT$(a_k, \cdot)$ returns `valid`; consequently, PIBT(a_2, a_1) returns `valid`. □

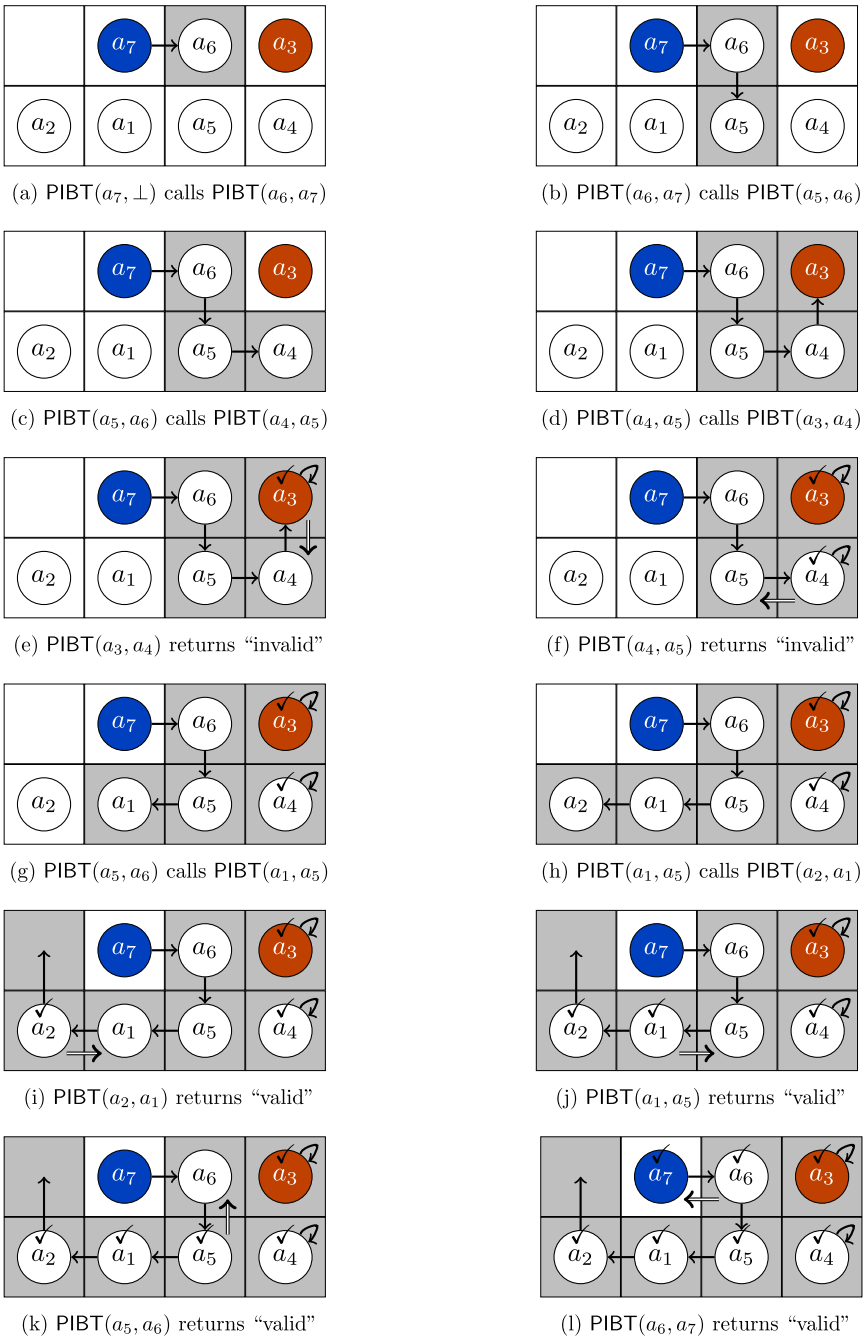


Fig. 3. Step-by-step example of Algorithm 1 for Fig. 2. A black arrow corresponds to $\pi_i[t+1]$ updated at either Line 14 or 20. A checked mark means that $\pi_i[t+1]$ has been fixed. A gray filled cell is a node requested as $\pi_j[t+1]$ from any agent a_j. A double-lined allow represents backtracking, corresponding to either Line 18 (valid) or 21 (invalid).

Theorem 2. *Given an MAPF instance such that G has a simple cycle for all pairs of adjacent nodes, PIBT (Algorithm 1) generates a set of conflict-free paths $\{\pi_1, \ldots, \pi_n\}$ such that for any agent $a_i \in A$, $\pi_i[0] = s_i$ and there is a timestep $t \leq diam(G) \cdot |A|$ such that $\pi_i[t] = g_i$.*

Proof. From Lemma 1, the agent with highest priority reaches its own goal within $diam(G)$ timesteps. Based on the update rule of the priority p_i for an agent a_i, once some agent a_i has reached its goal, p_i is reset and is lower than the priority of all other agents who have not reached their goal yet. These agents see their priority increase by one. As long as such agents remain, exactly one of them must have the highest priority. In turn, this agent reaches its own goal after at most $diam(G)$

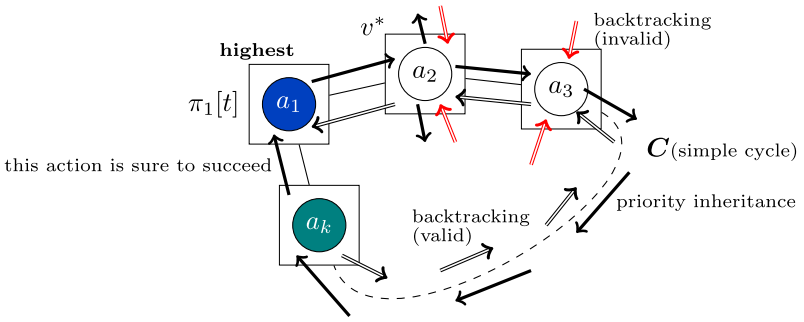
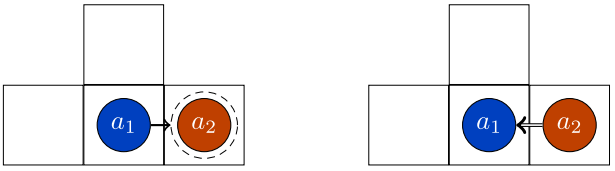


Fig. 4. Proof sketch of Lemma 1. The agent with the highest priority (a_1) can relocate to v^* if there exists a simple cycle $\boldsymbol{C} = (\pi_1[t], v^*, \ldots)$.



(a) Priority inheritance (b) Backtracking

Fig. 5. Failure case of PIBT on a graph without cycles. Assume that an agent a_1's goal is the location at another agent a_2. The priority inheritance to a_2 (5a) is "invalid" (5b) because a_2 has no escape node. Then, a_1 does replanning, however, it selects the current location as the next location since it is the next nearest vertex towards the goal. As a result, this situation will be held beyond the current timestep and there will be no progress.

timesteps. This repeats until all agents have reached their goal at least once, which takes at most $diam(G) \cdot |A|$ timesteps in total. □

Hereafter, this property is referred to as *reachability*, that is, all agents eventually reach their goal. Reachability differs from completeness for one-shot MAPF; it is never ensured that all agents reach their goal *simultaneously*; hence, PIBT is not ensured to solve conventional MAPF, defined in Section 3.1. Alternatively, with the graph condition of Theorem 2, PIBT is complete for the MAPF variant where agents need not necessarily stay at their goals.

A typical example that satisfies the aforementioned graph condition is the biconnected undirected graph. The opposite is not true, however, and Theorem 2 is expressed more generally to consider directed graphs. For instance, a directed ring satisfies the condition even though it is not biconnected.

Without the graph condition in Theorem 2, several agents may remain in the same vertices permanently and may not reach their goals. Fig. 5 shows such an example. In practice, PIBT needs a pre-defined maximum planning timestep to detect such failure cases. When PIBT reaches this timestep, it stops planning and reports a planning failure. See also Section 4.4.1.

4.3.2. Complexity analysis

Now we consider the time complexity of PIBT. Let F be the maximum time required for sorting candidate nodes [Line 10]. F depends on both G and the distance computation for the goals.

Proposition 3. *The time complexity of PIBT in one timestep is* $O\left(|A| \cdot (\Delta(G) + F + \lg|A|)\right)$.

Proof. Within one timestep, the procedure PIBT is called exactly $|A|$ times, once for each agent. This is because $\mathsf{PIBT}(a_i, \cdot)$ is called if and only if an agent a_i has not yet determined the next location (that is, when $\pi_i[t+1] = \bot$) [Lines 6, 16] but $\mathsf{PIBT}(a_i, \cdot)$ assigns $\pi_i[t+1]$ [Line 14] before calling another $\mathsf{PIBT}(a_k, a_i)$ for priority inheritance. The loop of Line 11–19 iterates at most $\Delta(G) + 1$ times. Each operation in the loop is performed in constant time. From the assumption, Line 10 requires $O(F)$. As a result, the procedure PIBT requires $O(|A| \cdot (\Delta(G) + F))$.

Line 3 requires $O(|A|)$. Line 4 requires $O(|A| \lg |A|)$. In total, PIBT in one timestep requires $O\left(|A| \cdot (\Delta(G) + F + \lg|A|)\right)$. □

The analysis is further continued for specific conditions. Assume that a distance table for the goal of each agent is computed by breadth-first search prior to executing PIBT, where the overhead is $O(|A| \cdot |E|)$. Then, F will be solely sorting candidate nodes following the table, resulting in $O(\Delta(G) \lg \Delta(G))$. Consequently, PIBT in one timestep is $O\left(|A| \cdot (\Delta(G) \lg \Delta(G) + \lg|A|)\right)$. Assume further that G is a 4-connected grid, as commonly used in MAPF studies. Here, instead of $\Delta(G)$, consider $\Delta(G) + 1$ for accurate analysis. Then, $(\Delta(G) + 1) \lg (\Delta(G) + 1) \simeq 11.6$. As a result, without a huge team of agents ($|A| < 3125$), the first term $|A| \cdot \Delta(G) \lg \Delta(G)$ is dominant; otherwise, the second term $|A| \lg |A|$ is dominant. In either case, *PIBT can be said to be computationally very inexpensive*.

The small time complexity provides several advantages. For instance, PIBT can address large instances for both A and G, which other MAPF algorithms cannot solve within a realistic timeframe. Another advantage is an application to *anytime planning*, a scheme that yields a feasible solution whenever interrupted but gradually improves solution quality as time goes by. Anytime planning is attractive, particularly in on-demand situations where the deliberation time is finite. A previous study realizes anytime planning for MAPF by refining known solutions iteratively [13]. For such an approach, obtaining initial solutions as quickly as possible is key because we can leave much time for refinement. In this sense, PIBT is a good choice to obtain an initial solution. In contrast, the anytime approach cannot be realized by "slow" solvers because they may not provide solutions within a time limit. Lastly, given a certain timestep, the runtime of PIBT is *predictable*, which is a fundamental characteristic in real-time planning. The power of PIBT is reflected in the experiment.

4.4. Application to specific problems

We now present how to adapt PIBT to solve typical pathfinding problems of multiple agents, namely, MAPF and MAPD.

4.4.1. Application to MAPF

For convenience, *configuration* refers to a set of locations of all agents at a given timestep. For instance, the initial configuration is $(s_1, \dots, s_n)$.

Termination So far, when the PIBT run has been completed, has not been specified. In the conventional MAPF, PIBT is assumed to run until it reaches the goal configuration $(g_1, \dots, g_n)$; otherwise, PIBT "fails" to solve the MAPF instance when it reaches the pre-defined timestep.

Distance evaluation In PIBT, for each timestep, each agent has to evaluate distances from the surrounding nodes to its goal [Line 10 of Algorithm 1]. This operation could be implemented by calling A* on demand but it could also be a bottleneck. Instead, PIBT can save computation time by preparing distance tables from each agent's goal. This is computed by breadth-first search from the goals with an overhead of $O(|A| \cdot |E|)$. Indeed, this is the main reason for the speedup of the implementation from our previous version of PIBT [21].

Extension for one-shot MAPF Even if G satisfies the condition for reachability, PIBT does not ensure that all agents reach their goal simultaneously, which is a requirement of conventional MAPF. In fact, with naive PIBT, a certain kind of livelock situation was confirmed in our experiment. This motivates the development of PIBT$^+$, a framework that enhances known MAPF solvers, presented in Algorithm 2. The main idea behind PIBT$^+$ is to make problem instances easier for the MAPF solver by bringing agents near their destinations using PIBT. In this sense, we say that the target MAPF solver is a *complement* solver.

Algorithm 2 PIBT$^+$.

Input: graph G, agents A, starts $\{s_1, \dots s_n\}$, goals $\{g_1, \dots g_n\}$
Output: paths $\boldsymbol{\pi} = \{\pi_1, \dots, \pi_n\}$
1: $T_{\min} \leftarrow \max_{a_j \in A} dist(\pi_j[0], g_j)$
2: Set initial priorities p_i to agents in descending order of distance from their initial location to their goals by adjusting the tie-breaker ϵ_i at Line 2 in Algorithm 1
3: $C_t \leftarrow$ configuration at $T_{\min}$ in $\boldsymbol{\pi}$
4: $\boldsymbol{\pi} \leftarrow$ PIBT (Algorithm 1) until timestep $T_{\min}$
5: **if** $C_t = (g_1, \dots, g_n)$ **then return** $\boldsymbol{\pi}$
6: $\boldsymbol{\pi}' \leftarrow$ other MAPF algorithm (*complement solver*), taking C_t as the initial configuration
7: **return** a concatenation of $\boldsymbol{\pi}$ and $\boldsymbol{\pi}'$

The concept of Algorithm 2 is as follows. $T_{\min}$ is the minimum number of timesteps needed for solutions from the initial configuration to the goal configuration. Because PIBT can be regarded as prioritized planning, the agent with the longest initial distance reaches its goal following the shortest path by providing priorities according to the initial distance. This implies that there is a chance that all agents reach their goals at $T_{\min}$. The remaining problem is relatively easy compared to the original because most agents are expected to have already reached their goals.

Theorem 4. *PIBT$^+$ is complete for MAPF on undirected graphs as long as the completeness conditions of the complement solver are satisfied.*

Proof. Let C_s, C_t, C_g be the initial configuration, the configuration at $T_{\min}$ planned by Line 3 in Algorithm 2, and the goal configuration, respectively. There exists a solution from C_t to C_s because PIBT computes the plan from C_s to C_t. If the original problem is solvable, there exists a solution from C_s to C_g, meaning that, at least one solution exists from C_t to C_g. According to this assumption, PIBT$^+$ finally uses the complete solver from C_t to C_g, and the complete solver must return a solution. The above satisfies the statement. □

PIBT$^+$ is essentially helpful for situations where it is difficult to obtain solutions through a direct adaptation of the complement solver but where plausible solutions are sought in a short time. As a complement solver, it is desirable to use complete algorithms such as rule-based, search-based, or compiling-based approaches. We note that PIBT$^+$ does not guarantee a better solution nor finish planning faster than using the complement solver from the beginning. On the other hand, since the transition from the initial configuration to the configuration at $T_{\min}$ is reversible, the additional burdens of sum-of-cost and makespan from optimal ones can be, respectively, at most $2|A| \cdot T_{\min}$ and $2T_{\min}$. Similarly, the additional burden of time complexity can be $O(T_{\min} \cdot |A| \cdot (\Delta(G) + F + \log|A|))$ from the consequence of Proposition 3.

4.4.2. Applying to MAPD

The MAPD problem is a typical example of online and lifelong scenarios requiring task allocation. We here describe how PIBT adapts to MAPD.

To design a complete MAPD algorithm, we must guarantee two claims: (1) all non-assigned tasks are eventually assigned, and (2) all assigned tasks are eventually completed, i.e., a task-assigned agent must visit pickup and delivery locations in order. For simplification, consider assigning a task to an agent only when the agent is at the pickup location. The remaining work for the agent is to visit the delivery location.

PIBT (Algorithm 1) is convenient for the second claim due to the following reason. Recall that Lemma 1 states that, for each timestep, the agent with the highest priority can always move to an arbitrary neighbor node. Using this lemma, ensuring that a task-assigned agent completes the task in finite time is straightforward. This is achieved with a special prioritization scheme that satisfies the two conditions: (1) all task-assigned agents have higher priorities than free agents, and (2) every task-assigned agent eventually gets the highest priority and holds it until the task is completed.

The first claim, all non-assigned tasks are eventually assigned, is achieved by various approaches. We here take a simple approach, i.e., for each timestep, all free agents set their goals to the nearest pickup location of non-assigned tasks. Assume that every free agent eventually gets the highest priority among free agents and holds the highest until it reaches a goal (a pickup location). Then, this approach satisfies the first claim because all task-assigned agents eventually become free due to the second claim.

Algorithm 3 PIBT for MAPD.

Input: graph G, agents A, initial locations $\{\pi_1[0], \dots, \pi_n[0]\}$, task stream Γ
Output: task assignment and paths $\{\pi_1, \dots, \pi_n\}$
(repeat below for each timestep $t = 0, 1, \dots$ until all tasks are completed)
1: **for** $a_i \in A$ **do**
2: **if** a_i is assigned to a task $\tau = (u, v)$ **then** $g_i \leftarrow v$; **continue**
3: Let $\tau' = (u', v')$ be an unassigned task from Γ that minimizes $dist(\pi_i[t], u')$
4: **if** no such τ' exists **then**
5: $g_i \leftarrow \pi_i[t]$
6: **else if** $u' = \pi_i[t]$ **then**
7: assign τ' to a_i; remove τ from Γ; $g_i \leftarrow v'$
8: **else**
9: $g_i \leftarrow u'$
10: **end if**
11: **end for**
12: $\mathcal{S} \leftarrow \{\pi_1[t], \dots, \pi_n[t]\}$; $\mathcal{G} \leftarrow \{g_1, \dots, g_n\}$
13: $\pi_1[t+1], \dots, \pi_n[t+1] \leftarrow$ Algorithm 1 for one-timestep with starts $\mathcal{S}$ and goals $\mathcal{G}$, while adding a prioritization rule such that all task-assigned agents have higher ones than those of free agents
14: **for** each agent $a_i \in A$ assigned to a task $\tau = (v, u)$ **do**
15: **if** $\pi_i[t+1] = u$ **then** make a_i free ▷ τ is completed
16: **end for**

Algorithm 3 realizes the aforementioned concept to solve MAPD. For each timestep, the algorithm first assigns tasks while updating the agents' goals [Lines 1–11]. Then, the algorithm uses PIBT to plan locations for the next timestep [Lines 12–13]. As discussed earlier, for each timestep, Algorithm 3 prioritizes the task-assigned agents than free agents. As a secondary prioritization, it uses the original PIBT prioritization scheme (Line 3 of Algorithm 1) as it is.

The following theorem is a consequence of the aforementioned discussion and its reachability.

Theorem 5. *Algorithm 3 is complete for an MAPD problem when G has a simple cycle for all pairs of adjacent nodes.*

Comparison of our proposed approach with an existing complete approach We later compare Algorithm 3 with the TP algorithm [18] experimentally. Before that, we here provide a qualitative discussion as follows. TP is complete for MAPD in a limited situation. First, it requires locations that never be either pickup or delivery locations for any tasks, called non-task endpoints. The non-task endpoints are required at least for the number of agents. Then, for any two endpoints, a path must exist without traversing any other endpoints. Here, endpoints consist of non-task endpoints and candidates of pickup/delivery locations. Compared to TP, PIBT does not require such conditions and works in a wide range of situations as long as the graph condition is satisfied.

Other task allocation We took a simple and greedy task allocation process; however, more aggressive approaches are applicable, e.g., incorporating linear cost-optimal assignment [63] or bottleneck assignment [64]. Doing so can improve solution qualities but potentially requires more computation time.

4.5. Decentralized online planning

Here, the decentralization of PIBT implementations is briefly discussed. In particular, we focus on the *online planning* of PIBT, that is, agents repeat a set of single-timestep planning and move actions. Because PIBT is based on prioritized planning, adopting a decentralized context is realistic. The part of priority inheritance and backtracking is performed by the propagation of information. Further, PIBT relies on local interactions between agents, implying that agents are not required to know other agents' information far away from their current location. To illustrate this, the concept of *interacting agents*, a set of agents that must negotiate their planning, is introduced.

Interacting agents When two agents are located within two hops of each other's move, they may collide in the next timestep; then, they are said to be *directly interacting* in that timestep. For an agent a_i, a group of *interacting agents* $\mathcal{A}_i(t) \subseteq A$ is then defined by transitivity over direct interactions in timestep t. Note that, given a_i and $\mathcal{A}_i(t)$, for any other agent $a_j \in \mathcal{A}_i(t)$, we have $\mathcal{A}_j(t) = \mathcal{A}_i(t)$. Whenever the context is obvious, $\mathcal{A}$ is directly used.

Because agents belonging to different groups cannot affect each other, path planning, the negotiation process using priority inheritance and backtracking can effectively occur in parallel. As a result, theoretically speaking, PIBT can be decentralized, relying only on local interactions, that is, it is sufficient that two agents in close proximity talk directly and utilize multi-hop communication; however, note that the groups of interacting agents are fully identified prior to starting a PIBT process for one timestep.

Communication Assume that interacting agents are fully detected at each timestep. Then, we analyze communication complexity, i.e., how many messages are required in PIBT. In a decentralized context, PIBT requires agents to know others' priorities before deciding on the next nodes. Usually, this requires $|\mathcal{A}|^2$ communication between agents; however, the update rule of the priority p_i relaxes this effort, for instance, storing other agents' priorities and communicating only when the priority is reset. Therefore, the communication cost of PIBT mainly depends on the information propagation phase.

Next, consider this information propagation phase. In PIBT, communication between agents corresponds to calling PIBT (priority inheritance), and when the procedure returns a value (corresponding to backtracking). $\mathsf{PIBT}(a_i, \cdot)$ is never called twice within a timestep, as discussed in the analysis of time complexity. Moreover, each agent sends a backtracking message at most once in each timestep. Overall, the communication cost for PIBT at each timestep is linear for the number of agents, that is, $O(|A|)$. In reality, this can be even lower because the figures depend on the number of interacting agents $|\mathcal{A}|$, which can be much smaller than $|A|$.

4.6. Without rotations

In practice, in physical environments such as mobile robots, movements corresponding to "rotations" might be difficult to realize due to synchronization problems. A rotation is a set of adjacent agents moving along a circle within one timestep. Formally, a rotation occurs for a subset of agents $\{a_i, a_j, a_k, \dots, a_l\} \subseteq A$ during timestep t and $t+1$ when $\pi_i[t+1] = \pi_j[t] \wedge \pi_j[t+1] = \pi_k[t] \wedge \dots \pi_l[t+1] = \pi_i[t]$ is satisfied. We refer to a set of paths as *rotation-free* when there is no rotation in the paths. This part adapts PIBT to situations where rotations are prohibited.

PIBT does not require major changes even in a model without rotations. During the iteration for the candidate nodes of the next timestep [Lines 11–19], skip nodes resulting in rotations like Lines 12 or 13 to prevent collisions. We denote the corresponding Algorithm 1 as PIBT$^{\otimes}$. This variant outputs a set of rotation-free paths.

Lemma 6. *Let a_1 denote the agent with highest priority at timestep t and v^* the nearest node to g_1 neighboring $\pi_1[t]$. If $|A| < |V|$ and there exists a path between v^* to an arbitrary node without going through $\pi_1[t]$, PIBT$^{\otimes}$ assigns v^* to a_1 as $\pi_i[t+1]$.*

Proof. Following $|A| < |V|$, we can derive $\exists v' \notin \{\pi_i[t] \,|\, a_i \in A\}$. There exists a path $\boldsymbol{D}$ between v^* to v' without going through $\pi_1[t]$. The subsequent proof is almost identical to that of Lemma 1 and uses $\boldsymbol{D}$ instead of $\boldsymbol{C}$. □

Theorem 7. *Given an MAPF instance such that G is biconnected and $|A| < |V|$, PIBT$^{\otimes}$ generates a set of conflict-free and rotation-free paths $\{\pi_1, \dots, \pi_n\}$ such that, for any agent $a_i \in A$, $\pi_i[0] = s_i$ and there is a timestep $t \leq diam(G) \cdot |A|$ such that $\pi_i[t] = g_i$.*

Proof. The same proof procedure of the reachability (Theorem 2) can be applied using Lemma 6 instead of Lemma 1. □

Note that this theorem is restricted to biconnected graphs compared to the original theorem on reachability.

5. Evaluation

This section evaluates PIBT thoroughly to empirically demonstrate that PIBT is a quick and scalable MAPF algorithm with acceptable solution quality. PIBT is tested in MAPF (Section 5.1) and MAPD (Section 5.2), followed by a stress test for the number of agents (Section 5.3) and robot demonstration (Section 5.4). The simulator was developed in C++, and the experiments were run on a laptop with Intel Core i9 2.3 GHz CPU and 16 GB RAM. The code is available at https://kei18.github.io/pibt2. We also provide the experimental result of PIBT for MAPF in extremely dense situations in Appendix B of the Appendix.

5.1. Multi-agent path finding (MAPF)

5.1.1. Setup

Benchmark The MAPF benchmark [1], which includes a set of four-connected grids and start–goal pairs for agents, was used. Ten grids were first selected with different portfolios, e.g., size, sparseness, and complexity (see Fig. 11, 12 in the Appendix). For each grid, 25 "random scenarios" were used while increasing the number of agents by ten up to the maximum (1000 agents in most cases). Therefore, identical instances were tried for the solvers in all settings. Note that almost all selected grids did not satisfy the graph condition of reachability. The problem setting follows conventional MAPF, i.e., agents are eventually on their goals simultaneously.

Baselines The following algorithms were carefully picked for comparisons with PIBT in MAPF:

- **HCA*** [4] as standard prioritized planning [19]. Aiming at improving solution quality, we prioritized each agent so that those with more considerable distances from their starts to goals have higher priorities, following heuristics from [53].
- **EECBS** [15] as a state-of-the-art bounded search-based sub-optimal solver; the sub-optimality was set to 1.2. Note that EECBS is bounded sub-optimal for the sum-of-costs metric.
- **EECBS-M**: The adapted version of EECBS for the makespan metric.
- **Push and Swap (PS)** [14] as a standard rule-based approach: although PS is incomplete as pointed out in [40], it is the origin of many extended algorithms [56,40,57,59]; hence, PS was used.
- **PS**$^+$: The post-processing solutions were obtained by PS. PS allows at most one agent to move, resulting in terrible outcomes. Thus, the solutions were compressed while preserving temporal dependencies of the solutions, influenced by the techniques in [65], which allow multiple agents to move simultaneously.
- **CBS** [16] with improvement techniques [66–71,12] as a state-of-the-art search-based optimal solver. Note that CBS optimizes the sum-of-costs metric.
- **CBS-M**: The makespan optimal version of CBS instead of sum-of-costs.
- **BCP** [17,11] as a state-of-the-art compiling-based optimal solver, which is optimal for the sum-of-costs metric.

For EECBS, CBS, and BCP, the implementations coded by their respective authors were used.[2] When these implementations have parameter specifications, the default was used for each. The other solvers (HCA*, PS, and PS$^+$) were coded by the author, which is own-coded in C++.

Failure case An algorithm was regarded as having failed to solve an instance when either of the three conditions was met: (1) The algorithm reported failure. (2) The algorithm reached the runtime limit (30 seconds). This value was based on [1]. (3) The algorithm reached the makespan limit (2000 in *brc202d*; otherwise 1000). This rules out impractical MAPF outcomes. The values were set according to preliminary results.

Metrics The following four evaluation metrics were used, referring to other MAPF studies: (1) **success rate** over 25 instances, (2) **runtime**, (3) **sum-of-costs** divided by $\sum_{a_i \in A} dist(s_i, g_i)$, and (4) **makespan** divided by $\max_{a_i \in A} dist(s_i, g_i)$. The latter two assess the solution quality; smaller solutions are better, and a minimum of one solution is necessary. Although optimal costs are challenging to obtain, these scores work as a lower bound of sub-optimality. Note that CBS and BCP are optimal for the sum-of-costs metric whereas CBS-M is optimal for makespan; we hence omit to plot makespan scores of CBS/BCP and sum-of-costs scores of CBS-M. Sum-of-costs and makespan have Pareto optimal structure [7]; hence, generally, they cannot be minimized simultaneously.

Other remarks PIBT$^+$ uses PS$^+$ as a complement solver. Own-coded solvers (PIBT$^{(+)}$, HCA*, PS$^{(+)}$) used a distance table for the start locations of each agent, computed by breadth-first search. The runtime included the procedure of creating the table.

[2] The codes are available on https://github.com/Jiaoyang-Li/EECBS, https://github.com/Jiaoyang-Li/CBSH2-RTC, and https://github.com/ed-lam/bcp-mapf, respectively. (EE)CBS-M was implemented in a best-first search manner that prioritizes makespan-better search nodes in the high-level tree, modifying the aforementioned codes.

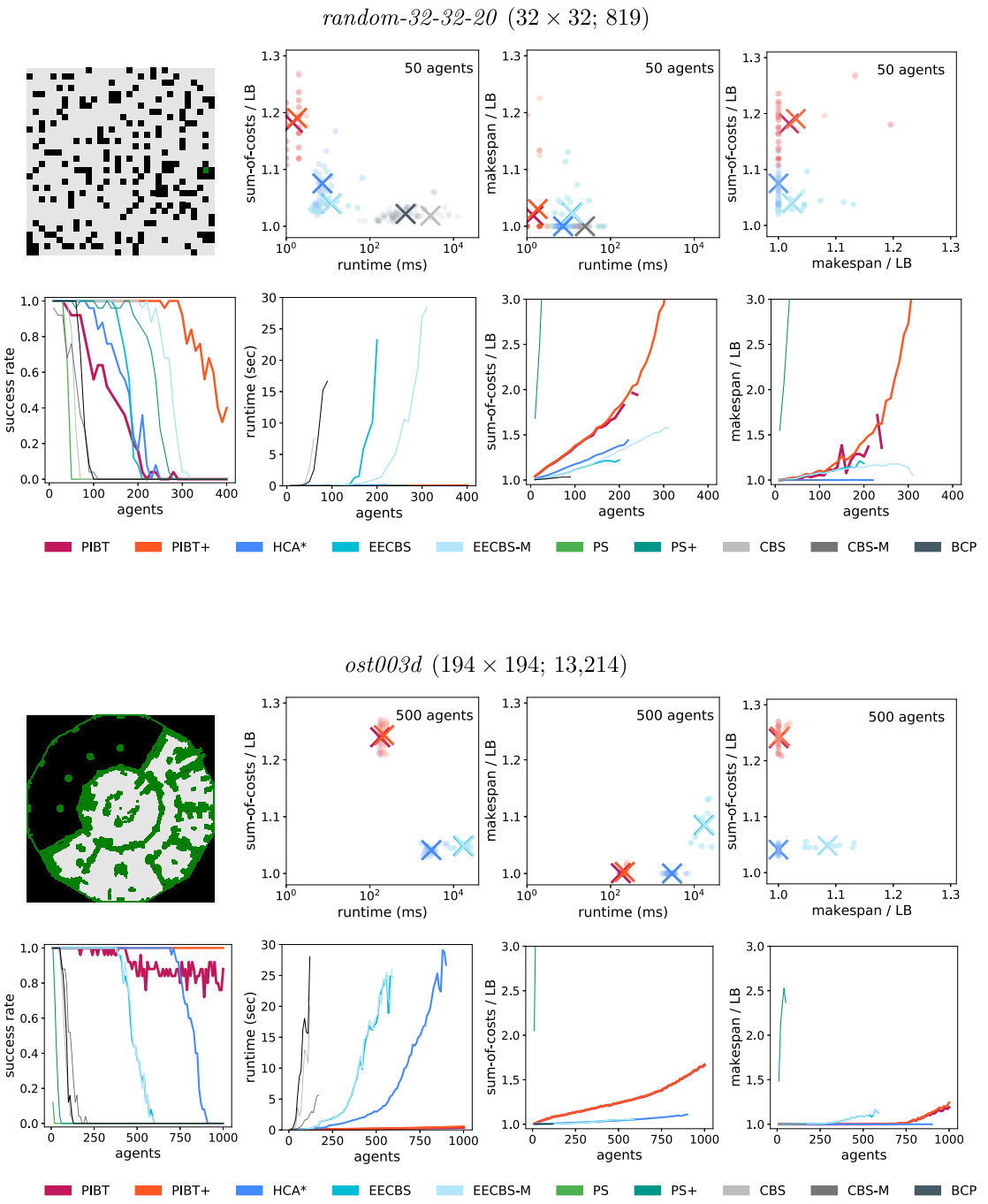


Fig. 6. Part of MAPF results. As typical examples, *random-32-32-20* and *ost003d* were selected. The complete results are available in the Appendix. In the upper-right part, charts with a fixed number of agents are shown. The map size and $|V|$ are shown with parentheses. The scores of sum-of-costs for CBS-M and those of makespan for CBS/BCP are omitted. In the upper rightmost chart of *ost003d*, the plots of EECBS and EECBS-M are overlapped.

5.1.2. Result

Fig. 6 presents part of the results. The full version is displayed in Fig. 11 and 12 in the Appendix. Excluding success rates, the scores are averaged over only the successful instances with that solver.[3] In addition, we show the classification of failures of each solver in Table 1.

[3] For a fair comparison, it is better to present the average scores of instances that all solvers solved. However, due to the high failure rates, the current style of data plots was selected.

Table 1
Classification of failures. The numbers of failed instances, regardless of $|A|$, are reported.

random-32-32-20

	PIBT	PIBT$^+$	HCA*	EECBS	EECBS-M	PS	PS$^+$	CBS	CBS-M	BCP
total	674	102	590	568	333	908	426	872	862	823
1. failure report	0	3	590	0	0	0	0	0	0	0
2. makespan limit	674	99	0	0	0	908	426	0	0	0
3. time over	0	0	0	568	333	0	0	872	862	823

ost003d

	PIBT	PIBT$^+$	HCA*	EECBS	EECBS-M	PS	PS$^+$	CBS	CBS-M	BCP
total	204	0	473	1301	1296	2497	2439	2311	2237	2294
1. failure report	0	0	0	0	0	0	0	0	0	0
2. makespan limit	204	0	0	0	0	2497	2439	0	0	0
3. time over	0	0	473	1301	1296	0	0	2311	2237	2294

Overall, PIBT outputs solutions immediately ($\leq$ 5 seconds in large maps) even with a thousand agents, with a slight decline in the solution quality (sum-of-costs). Specific findings are summarized as follows:

PIBT is extremely scalable The scalability here is defined by map size and the number of agents. In both aspects, PIBT outperforms the other baselines. The runtime of PIBT is significantly faster by orders of magnitude compared to the other solvers. In other words, PIBT can solve large instances that other solvers cannot solve. This is owing to the small-time complexity.

PIBT outputs solutions with acceptable quality in sparse situations In general, PIBT does not guarantee solution quality. However, both sum-of-costs and makespan are adequate compared to others in sparse settings. This is in contrast to pure rule-based solvers (PS$^{(+)}$) that mostly fail due to the makespan limit even with fewer agents. PIBT is based on prioritized planning, enabling output solutions with acceptable quality.

Failure reasons of PIBT Table 1 reveals that the failure reason for PIBT is due to the makespan limit. We further explain two failure categories of PIBT as follows. The first case is due to the nature of reachability (Theorem 2) that does not ensure that all agents reach their goals simultaneously. The second case is due to graphs without a cycle for any two adjacent nodes. In such graphs, it is possible to reach situations where several agents stop moving, similar to Fig. 5. As those agents remain in their locations, PIBT eventually reaches the makespan limit. Therefore, PIBT often fails with many agents in small maps (for instance, *empty-48-48* and *random-64-64-20*).

PIBT$^+$ can increase the success rate of PIBT dramatically The aforementioned shortcoming of PIBT for one-shot MAPF is overcome by adding a complement solver. Note that PIBT$^+$ sometimes failed because an incomplete solver PS$^+$ was used as a complement solver. Note further that it is possible to use other solvers for the complement solver.

Solution quality of PIBT$^+$ in dense situations In Fig. 6, the upper bound of suboptimality of PIBT$^+$ in *random-32-32-20* dramatically increases with the number of agents. This is due to the complement solver. In the experiment, PIBT$^+$ used PS$^+$ as the complement solver to achieve high success rates within a small computation time. In general, PS$^+$ is quick even for large instances, however, its solution quality is not excellent, as seen in our results. In cluttered and dense situations such as *random-32-32-20*, it is difficult to obtain solutions by PIBT. Then, PIBT$^+$ often uses the complement solver, degrading the solution quality.

Interpretations of comparisons' results Optimal solvers (CBS and BCP) can address a few hundred agents in some cases, but usually, they cannot address several hundreds of agents, emphasizing the necessity of sub-optimal solvers. Pure rule-based solvers (PS$^{(+)}$) are quick, but their solution qualities are not acceptable compared to the other solvers. Indeed, failures of PS$^{(+)}$ are mainly due to the makespan limit. Prioritized planning (HCA*) outputs plausible solutions for both sum-of-costs and makespan. It is scalable but not so much as PIBT. In addition, when scenarios are dense, it is significantly difficult to find solutions according to *static* priorities (see *random-32-32-20* in Table 1), making *dynamic* priorities attractive as used in PIBT. The overall result of the search-based sub-optimal solver (EECBS) is similar to HCA*, i.e., it outputs plausible solutions but is not as quick and scalable as PIBT. In short, PIBT has a unique position compared to other established MAPF approaches, i.e., it is rapid and scalable with acceptable solution qualities.

5.2. Multi-agent pickup and delivery (MAPD)

5.2.1. Setup

The experimental setup follows that of the original MAPD study [18], using the same undirected graph as a testbed, and setting the same locations as candidates for pickup and delivery (Fig. 7). A sequence of 500 tasks was generated by

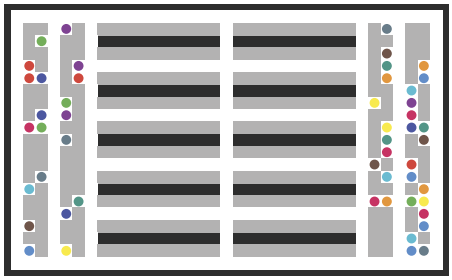


Fig. 7. Map used in the MAPD experiment. The graph is 21×35 4-connected grid. Gray cells in the MAPD map are task endpoints, i.e., pickup and delivery locations.

randomly choosing pickup and delivery locations from all task endpoints. The six different task frequencies where numbers of tasks are added to the task set Γ were as follows: 0.2 (one task every five timesteps), 0.5, 1, 2, 5, and 10 with the number of agents increasing from 10 to 50. The Token Passing (TP) algorithm [18] was also tested as a baseline, which we programmed in C++. All experimental settings were performed over 100 instances in which initial positions of agents were set randomly. The following three metrics were evaluated: (1) **runtime** per one timestep; MAPD is assumed to be online, (2) **service time**: from issue to completion of tasks, and (3) **makespan**: the first timestep at which all tasks are completed. Both PIBT (Algorithm 3) and TP used an all-pairs distance matrix, pre-computed with the Floyd–Warshal algorithm [72].

5.2.2. Result

Fig. 8 summarizes the results. PIBT significantly outperforms TP in runtime and is comparable to or better than TP in solution quality, characterized by service time and makespan. The runtime improvements are because of the low time complexity of PIBT. The improvements in solution quality are mainly due to how the algorithm deals with free agents. Assume that there is one unassigned task. In PIBT, all free agents move toward the pickup location, with only the earliest one actually getting the task and then having to start moving to the delivery location while *pushing the other free agents away*, thanks to the prioritization scheme. In contrast, TP evacuates all free agents to non-task endpoints to avoid deadlocks other than the one agent that is newly assigned. However, this assigned agent must still *"dodge" those free agents' locations* to prevent deadlocks. As a result, the path planned by PIBT for the task-assigned agent will be shorter than that of TP.

5.3. Stress test for scalability

5.3.1. Setup

The scalability of PIBT was evaluated through MAPF with a large map while varying the number of agents from $2,000$ to $10,000$. The map, shown in Fig. 9, was the largest in the MAPF benchmark [1]. Here, only runtime was evaluated per timestep, averaged over the first 100 timesteps, regardless of whether the planning succeeded. Similar to the previous MAPF experiment, distance tables computed before executing PIBT were used. The starts and goals of agents were set randomly for each repetition.

5.3.2. Result

Fig. 9 summarizes the result. The scores follow almost a linear trend in the number of agents. Surprisingly, even with $10,000$ agents, PIBT scored around ten milliseconds per one timestep on an ordinary laptop.

5.4. Demonstrations with real robots

PIBT was also implemented with a team of small physical robots. Here, an online and lifelong scenario is presented where a new goal is immediately assigned to an agent who reaches its current goal. Note that, because of reachability, PIBT ensures that all assigned goals are eventually met, that is, visited by assigned robots.

Platform *Toio* robots (https://toio.io/) were used to implement PIBT. The robots, connected to a master computer via the Bluetooth LE (low energy) protocol, advance on a specific playmat and are controllable by instructions using absolute coordinates.

Usage The robots were controlled in a *centralized*, *synchronized*, and *online* mode, described as follows. A virtual grid (7×5 with obstacles) was created on the playmat; the robots followed the grid. A central server (a laptop) managed the locations of all robots. Periodically, the server executed PIBT for one single timestep and issued the instructions (that is, where to go) to each robot. The code was written in Node.js.

Snapshot Fig. 10 shows a snapshot of the demo. The full video and code are also available at https://kei18.github.io/pibt2.

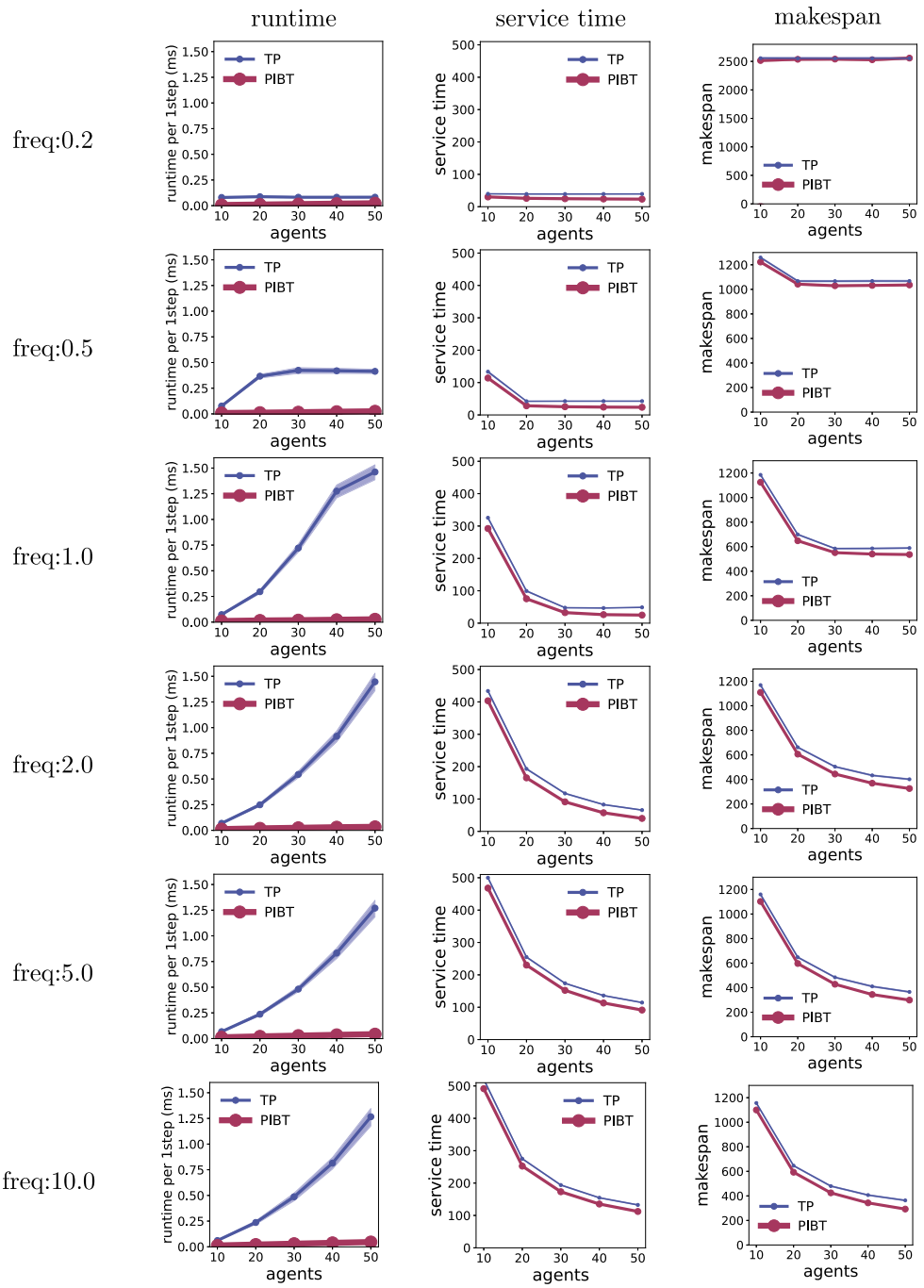
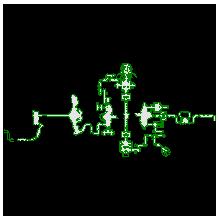


Fig. 8. Summary of MAPD results. Each value is an average of 100 instances with 95% confidence intervals. For both service time and makespan, the intervals are hard to recognize because they are tiny.

6. Conclusion

This study introduces PIBT, a scalable algorithm for solving MAPF iteratively. PIBT focuses on the adjacent movements of multiple agents, relying on a simple prioritization scheme; hence, it can be applied to several domains, including online and lifelong situations. The four empirical results support this aspect: (1) PIBT$^{(+)}$ can promptly solve large MAPF instances, whereas other solvers take the time or require unrealistic computational times. (2) PIBT outperforms current solutions for pickup and delivery (MAPD). (3) Despite thousands of agents, PIBT can yield planning in a real-time manner. (4) A real-robot

orz900d
1491×656 $(96,603)$



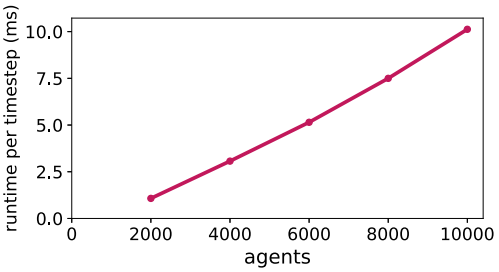


Fig. 9. The results of the stress test. The average runtime per timestep is shown. The plots of the 95% confidence intervals are too small and difficult to recognize.

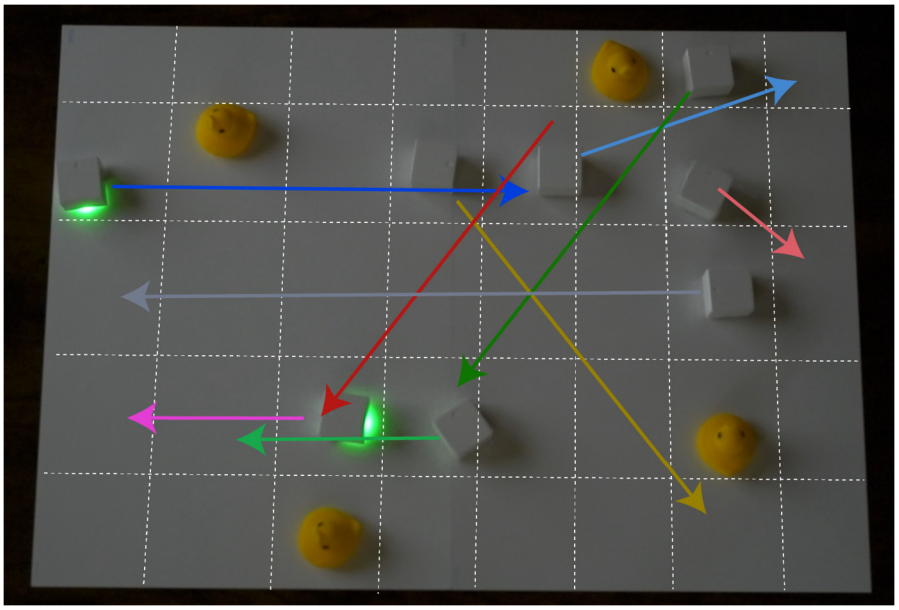


Fig. 10. Snapshot of PIBT demo with real robots. The virtual gird is shown with white dotted lines. Each robot's goal is further annotated with colored arrows. Green-lighted robots have recently been allocated. Yellow ducks are obstacles. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

execution of PIBT was further presented. We believe that PIBT provides practical insights into controlling a swarm of agents in practical situations.

Future directions for research in this field include jumping into the "wild" environment, that is, considering *asynchrony*. In traditional MAPF settings, movement between agents is assumed to be synchronized. Although this assumption enables the agents' behavior control, there exists a significant gap in practical situations mainly because of the asynchrony of the agents' moves, which requires further consideration. Our recent study [73] presents other directions.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

We thank the anonymous reviewers for their many insightful comments. This research is partly supported by JSPS KAKENHI Grant Numbers 20J23011, 21K11748, and 21H03423. Keisuke Okumura would like to thank the Yoshida Scholarship Foundation for their support. We thank Editage (www.editage.com) for English language editing.

Appendix A. MAPF results

Figs. 11 and 12 summarize all results of the MAPF experiments in Section 5.1.

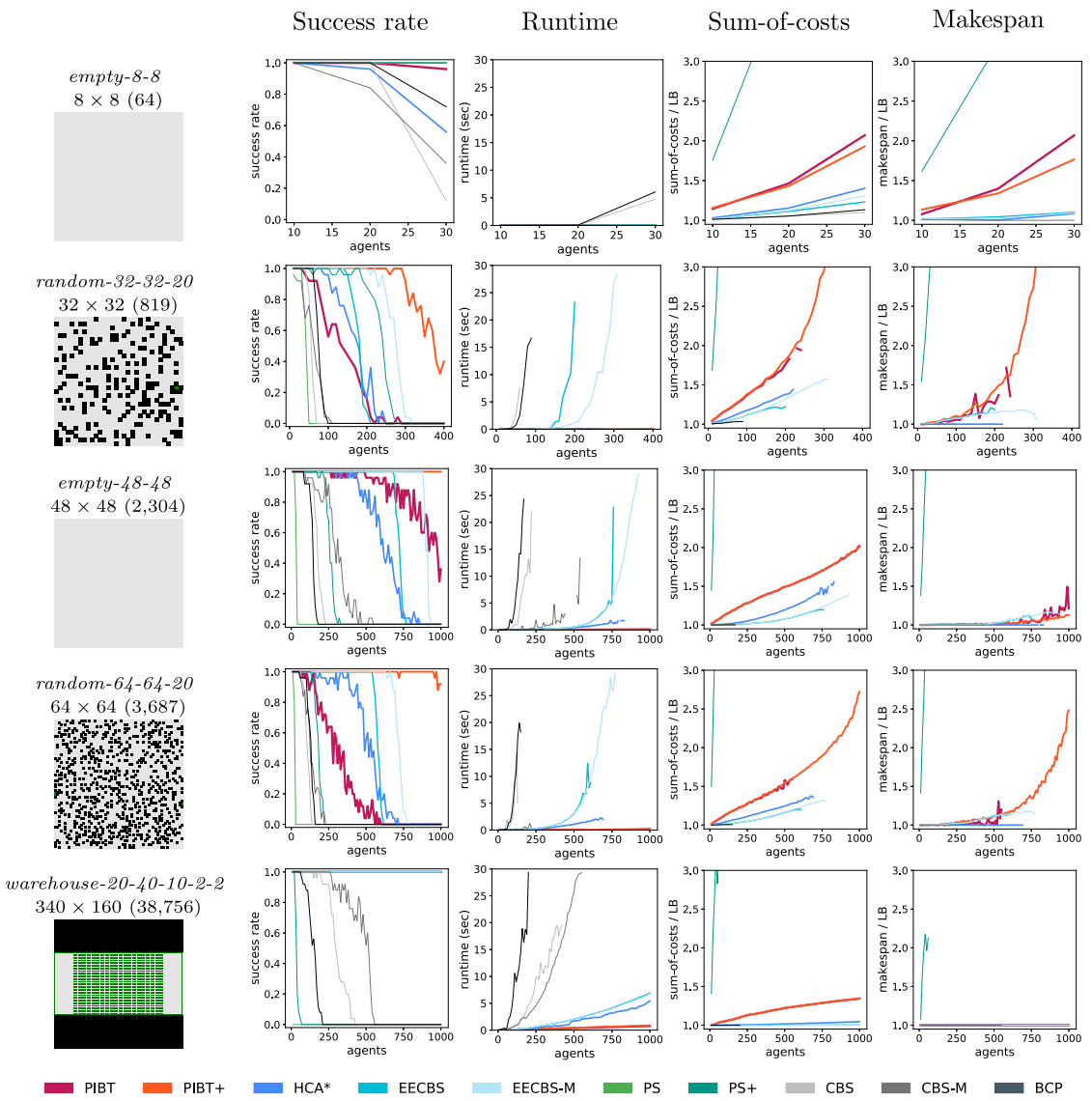


Fig. 11. Summary of MAPF results (1/2). $|V|$ is shown in parentheses.

Table 2
PIBT in extremely dense situation. The used map is *empty-8-8*. 25 instances were prepared for each $|A|$. Scores of "sum-of-costs" and "makespan" are upper bounds of sub-optimality in the same way as Fig. 6.

$\|A\|$	success rate	runtime (ms)	sum-of-costs	makespan
40	0.96	0.21	3.15	3.46
50	0.84	1.43	7.38	6.94
60	1.00	2.16	12.25	7.86
64	1.00	3.16	21.55	10.01

Appendix B. PIBT in extremely dense situations

This section presents additional experiments on PIBT in extremely dense situations where $|A| > |V|/2$ and G satisfies the graph condition of Theorem 2. We used *empty-8-8* and the same experimental settings as those introduced in Section 5.1.

Table 2 summarizes the result. Most instances are solved at most 5 ms; otherwise, PIBT failed by reaching the makespan limit. Solution qualities dramatically increase with more agents because, in dense situations, most agents cannot take their shortest paths.

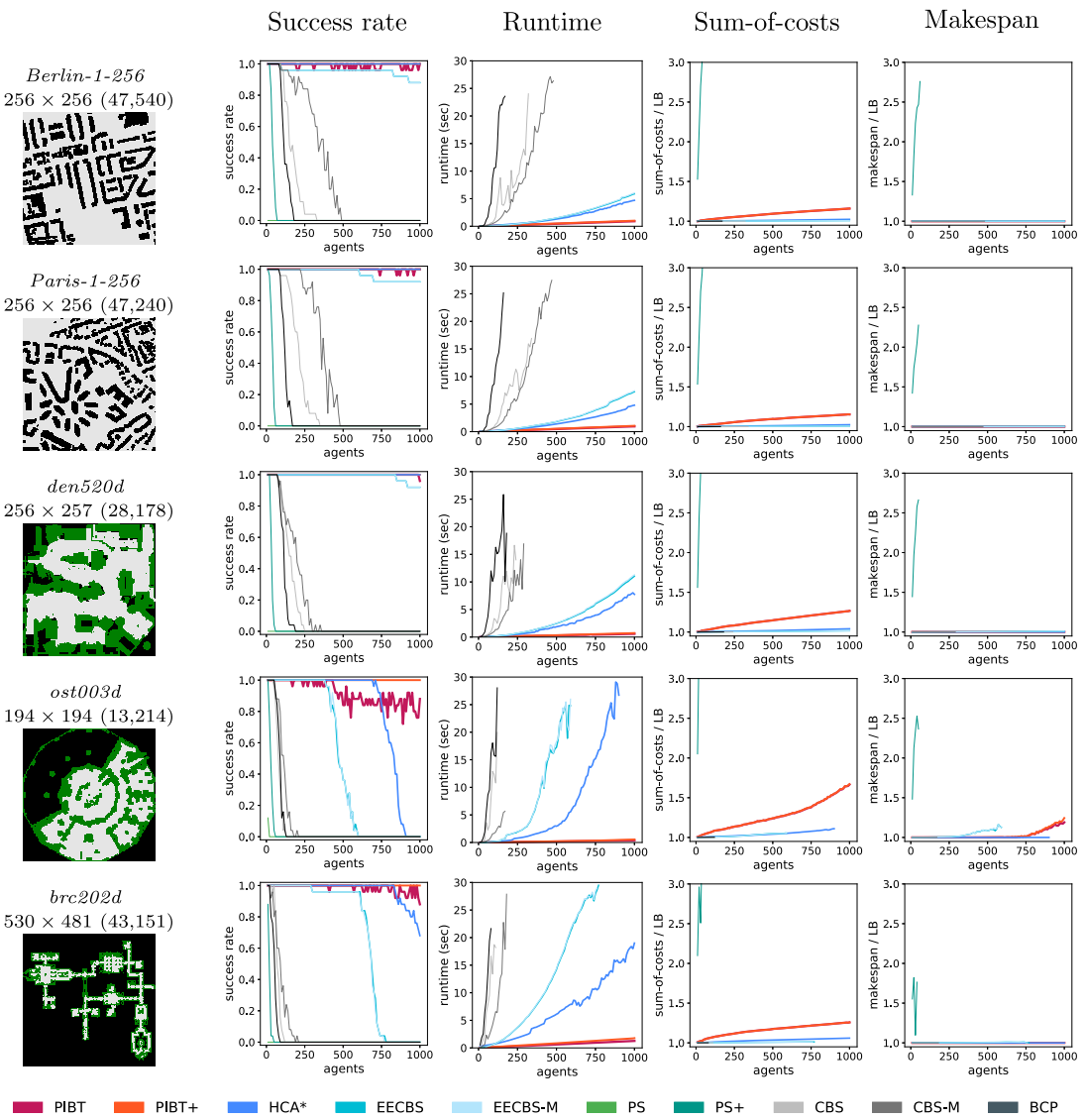


Fig. 12. Summary of MAPF results (2/2). $|V|$ is shown in parentheses.

Interestingly, PIBT solved all instances with 64 agents (fully occupied), while sometimes failing with 50 agents. This is because of the randomness of the tie-break rule at Line 10 of Algorithm 1. As a discussion of the implementation level, we used three rules when sorting candidate nodes for the following location: (1) distances to the goal, (2) the presence of agents to avoid unnecessary priority inheritance, and (3) random values, when neither the prior two break a tie. We also tested the rule-2 omitted version, succeeding for all instances regardless of $|A|$. Since the rule-2 loses its meaning in situations where $|A| = |V|$, we consider that the randomness contributes to solving extremely dense situations. We also observed that this trend is the same when testing a 16×16 empty grid, i.e., usual PIBT sometimes failed whereas PIBT that omits the rule-2 tie-break solved all instances regardless of $|A|$. From these observations, we guess that in tidy environments like *empty-8-8*, PIBT without the rule-2 can solve one-shot MAPF with a high probability as makespan increases. This is an interesting direction to seek but significantly beyond this paper.

Note that the aforementioned discussion is not applicable when G does not satisfy the graph condition of Theorem 2. Moreover, rule-2 is usually effective in improving sum-of-costs, as shown in Table 3.

References

[1] R. Stern, N. Sturtevant, A. Felner, S. Koenig, H. Ma, T. Walker, J. Li, D. Atzmon, L. Cohen, T. Kumar, et al., Multi-agent pathfinding: definitions, variants, and benchmarks, in: Proceedings of Annual Symposium on Combinatorial Search (SOCS), 2019.
[2] K. Dresner, P. Stone, A multiagent approach to autonomous intersection management, J. Artif. Intell. Res. (2008).
[3] P.R. Wurman, R. D'Andrea, M. Mountz, Coordinating hundreds of cooperative, autonomous vehicles in warehouses, AI Mag. (2008).

Table 3
Effect of tie-break strategy. "normal" is PIBT and "random" is PIBT without the rule-2 tie-break. The used map was *den520d*. For each $|A|$, 25 random scenarios were prepared, not equivalent to those in Sec. 5.1. The scores of sum-of-costs and makespan are average of upper bounds of sub-optimality with 95% confidence intervals, based on instances that were solved by both.

	success rate		sum-of-costs		makespan	
$\|A\|$	normal	random	normal	random	normal	random
100	1.00	1.00	1.04 (1.04, 1.05)	1.08 (1.07, 1.09)	1.00 (1.00, 1.00)	1.00 (1.00, 1.00)
300	1.00	1.00	1.10 (1.09, 1.10)	1.16 (1.15, 1.16)	1.00 (1.00, 1.00)	1.00 (1.00, 1.00)
500	0.96	1.00	1.15 (1.14, 1.15)	1.22 (1.22, 1.23)	1.00 (1.00, 1.00)	1.00 (1.00, 1.00)
700	0.96	0.96	1.20 (1.20, 1.20)	1.28 (1.27, 1.28)	1.00 (1.00, 1.00)	1.00 (1.00, 1.00)
900	0.88	0.96	1.25 (1.24, 1.25)	1.33 (1.32, 1.34)	1.00 (1.00, 1.00)	1.00 (1.00, 1.01)

[4] D. Silver, Cooperative pathfinding, in: Proceedings of AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE), 2005.
[5] R. Morris, C.S. Pasareanu, K.S. Luckow, W. Malik, H. Ma, T.S. Kumar, S. Koenig, Planning, scheduling and monitoring for airport surface operations, in: Proceedings of AAAI Workshop: Planning for Hybrid Systems, 2016.
[6] A. Okoso, K. Otaki, T. Nishi, Multi-agent path finding with priority for cooperative automated valet parking, in: Proceedings of IEEE Intelligent Transportation Systems Conference (ITSC), 2019.
[7] J. Yu, S.M. LaValle, Structure and intractability of optimal multi-robot path planning on graphs, in: Proceedings of AAAI Conference on Artificial Intelligence (AAAI), 2013.
[8] H. Ma, C. Tovey, G. Sharon, T.S. Kumar, S. Koenig, Multi-agent path finding with payload transfers and the package-exchange robot-routing problem, in: Proceedings of AAAI Conference on Artificial Intelligence (AAAI), 2016.
[9] J. Yu, Intractability of optimal multirobot path planning on planar graphs, IEEE Robot. Autom. Lett. (2015).
[10] J. Banfi, N. Basilico, F. Amigoni, Intractability of time-optimal multirobot path planning on 2d grid graphs with holes, IEEE Robot. Autom. Lett. (2017).
[11] E. Lam, P. Le Bodic, New valid inequalities in branch-and-cut-and-price for multi-agent path finding, in: Proceedings of International Conference on Automated Planning and Scheduling (ICAPS), 2020.
[12] J. Li, D. Harabor, P.J. Stuckey, S. Koenig, Pairwise symmetry reasoning for multi-agent path finding search, Artif. Intell. (2021).
[13] K. Okumura, Y. Tamura, X. Défago, Iterative refinement for real-time multi-robot path planning, in: Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2021.
[14] R. Luna, K.E. Bekris, Push and swap: fast cooperative path-finding with completeness guarantees, in: Proceedings of International Joint Conference on Artificial Intelligence (IJCAI), 2011.
[15] J. Li, W. Ruml, S. Koenig, Eecbs: a bounded-suboptimal search for multi-agent path finding, in: Proceedings of AAAI Conference on Artificial Intelligence (AAAI), 2021.
[16] G. Sharon, R. Stern, A. Felner, N.R. Sturtevant, Conflict-based search for optimal multi-agent pathfinding, Artif. Intell. (2015).
[17] E. Lam, P. Le Bodic, D.D. Harabor, P.J. Stuckey, Branch-and-cut-and-price for multi-agent pathfinding, in: Proceedings of International Joint Conference on Artificial Intelligence (IJCAI), 2019.
[18] H. Ma, J. Li, T. Kumar, S. Koenig, Lifelong multi-agent path finding for online pickup and delivery tasks, in: Proceedings of International Joint Conference on Autonomous Agents & Multiagent Systems (AAMAS), 2017.
[19] M. Erdmann, T. Lozano-Perez, On multiple moving objects, Algorithmica (1987).
[20] L. Sha, R. Rajkumar, J.P. Lehoczky, Priority inheritance protocols: an approach to real-time synchronization, IEEE Trans. Comput. (1990).
[21] K. Okumura, M. Machida, X. Défago, Y. Tamura, Priority inheritance with backtracking for iterative multi-agent path finding, in: Proceedings of International Joint Conference on Artificial Intelligence (IJCAI), 2019.
[22] P. Surynek, An optimization variant of multi-robot path planning is intractable, in: Proceedings of AAAI Conference on Artificial Intelligence (AAAI), 2010.
[23] D.M. Kornhauser, G. Miller, P. Spirakis, Coordinating pebble motion on graphs, the diameter of permutation groups, and applications, Master's thesis, MIT, 1984.
[24] B. Nebel, On the computational complexity of multi-agent pathfinding on directed graphs, in: Proceedings of International Conference on Automated Planning and Scheduling (ICAPS), 2020.
[25] A. Botea, D. Bonusi, P. Surynek, Solving multi-agent path finding on strongly biconnected digraphs, J. Artif. Intell. Res. (2018).
[26] T.S. Standley, Finding optimal solutions to cooperative pathfinding problems, in: Proceedings of AAAI Conference on Artificial Intelligence (AAAI), 2010.
[27] G. Sharon, R. Stern, M. Goldenberg, A. Felner, The increasing cost tree search for optimal multi-agent pathfinding, Artif. Intell. (2013).
[28] M. Goldenberg, A. Felner, R. Stern, G. Sharon, N. Sturtevant, R. Holte, J. Schaeffer, Enhanced partial expansion A*, J. Artif. Intell. Res. (2014).
[29] G. Wagner, H. Choset, Subdimensional expansion for multirobot path planning, Artif. Intell. (2015).
[30] M. Barer, G. Sharon, R. Stern, A. Felner, Suboptimal variants of the conflict-based search algorithm for the multi-agent pathfinding problem, in: Proceedings of Annual Symposium on Combinatorial Search (SOCS), 2014.
[31] P. Surynek, Towards optimal cooperative path planning in hard setups through satisfiability solving, in: Proceedings of Pacific Rim International Conference on Artificial Intelligence (PRICAI), 2012.
[32] P. Surynek, A. Felner, R. Stern, E. Boyarski, Efficient sat approach to multi-agent path finding under the sum of costs objective, in: Proceedings of European Conference on Artificial Intelligence (ECAI), 2016.
[33] P. Surynek, Unifying search-based and compilation-based approaches to multi-agent path finding through satisfiability modulo theories, in: Proceedings of International Joint Conference on Artificial Intelligence (IJCAI), 2019.
[34] J. Yu, S.M. LaValle, Optimal multirobot path planning on graphs: complete algorithms and effective heuristics, IEEE Trans. Robot. (2016).
[35] E. Erdem, D.G. Kisa, U. Öztok, P. Schüller, A general formal framework for pathfinding problems with multiple agents, in: Proceedings of AAAI Conference on Artificial Intelligence (AAAI), 2013.
[36] R.N. G'omez, C. Hern'andez, J.A. Baier, Solving sum-of-costs multi-agent pathfinding with answer-set programming, in: Proceedings of AAAI Conference on Artificial Intelligence (AAAI), 2020.
[37] M.R.K. Ryan, Exploiting subgraph structure in multi-robot path planning, J. Artif. Intell. Res. (2008).
[38] M. Peasgood, C.M. Clark, J. McPhee, A complete and scalable strategy for coordinating multiple robots within roadmaps, IEEE Trans. Robot. (2008).

[39] P. Surynek, A novel approach to path planning for multiple robots in bi-connected graphs, in: Proceedings of IEEE International Conference on Robotics and Automation (ICRA), 2009.
[40] B. de Wilde, A.W. ter Mors, C. Witteveen, Push and rotate: cooperative multi-agent path planning, in: Proceedings of International Joint Conference on Autonomous Agents & Multiagent Systems (AAMAS), 2013.
[41] G. Sartoretti, J. Kerr, Y. Shi, G. Wagner, T.S. Kumar, S. Koenig, H. Choset, Primal: pathfinding via reinforcement and imitation multi-agent learning, IEEE Robot. Autom. Lett. (2019).
[42] M. Damani, Z. Luo, E. Wenzel, G. Sartoretti, Primal_2: pathfinding via reinforcement and imitation multi-agent learning-lifelong, IEEE Robot. Autom. Lett. (2021).
[43] Q. Li, F. Gama, A. Ribeiro, A. Prorok, Graph neural networks for decentralized multi-robot path planning, in: Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2020.
[44] Q. Li, W. Lin, Z. Liu, A. Prorok, Message-aware graph attention networks for large-scale multi-robot path planning, IEEE Robot. Autom. Lett. (2021).
[45] A. Felner, R. Stern, S.E. Shimony, E. Boyarski, M. Goldenberg, G. Sharon, N. Sturtevant, G. Wagner, P. Surynek, Search-based optimal solvers for the multi-agent pathfinding problem: summary and challenges, in: Proceedings of Annual Symposium on Combinatorial Search (SOCS), 2017.
[46] R. Stern, Multi-agent path finding–an overview, in: Artificial Intelligence (AIJ), 2019.
[47] K.-H.C. Wang, A. Botea, Mapp: a scalable multi-agent path planning algorithm with tractability and completeness guarantees, J. Artif. Intell. Res. (2011).
[48] P. Velagapudi, K. Sycara, P. Scerri, Decentralized prioritized planning in large multirobot teams, in: Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2010.
[49] Y. Jiang, H. Yedidsion, S. Zhang, G. Sharon, P. Stone, Multi-robot planning with conflicts and synergies, Auton. Robots (2019).
[50] K. Azarm, G. Schmidt, Conflict-free motion of multiple mobile robots based on decentralized motion planning and negotiation, in: Proceedings of IEEE International Conference on Robotics and Automation (ICRA), 1997.
[51] Z. Bnaya, A. Felner, Conflict-oriented windowed hierarchical cooperative A*, in: Proceedings of IEEE International Conference on Robotics and Automation (ICRA), 2014.
[52] M. Bennewitz, W. Burgard, S. Thrun, Finding and optimizing solvable priority schemes for decoupled path planning techniques for teams of mobile robots, Robot. Auton. Syst. (2002).
[53] J.P. Van Den Berg, M.H. Overmars, Prioritized motion planning for multiple robots, in: Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2005.
[54] M. Čáp, P. Novák, A. Kleiner, M. Seleckỳ, Prioritized planning algorithms for trajectory coordination of multiple mobile robots, IEEE Trans. Autom. Sci. Eng. (2015).
[55] H. Ma, D. Harabor, P.J. Stuckey, J. Li, S. Koenig, Searching with consistent prioritization for multi-agent path finding, in: Proceedings of AAAI Conference on Artificial Intelligence (AAAI), 2019.
[56] Q. Sajid, R. Luna, K.E. Bekris, Multi-agent pathfinding with simultaneous execution of single-agent primitives, in: Proceedings of Annual Symposium on Combinatorial Search (SOCS), 2012.
[57] A. Wiktor, D. Scobee, S. Messenger, C. Clark, Decentralized and complete multi-robot motion planning in confined spaces, in: Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2014.
[58] C. Wei, K.V. Hindriks, C.M. Jonker, Multi-robot cooperative pathfinding: a decentralized approach, in: Proceedings of International Conference on Industrial, Engineering and Other Applications of Applied Intelligent Systems, 2014.
[59] Y. Zhang, K. Kim, G. Fainekos, Discof: cooperative pathfinding in distributed systems with limited sensing and communication range, in: Proceedings of International Symposium on Distributed Robotic Systems (DARS), 2016.
[60] M.M. Khorshid, R.C. Holte, N.R. Sturtevant, A polynomial-time algorithm for non-optimal multi-agent pathfinding, in: Proceedings of Annual Symposium on Combinatorial Search (SOCS), 2011.
[61] J. Švancara, M. Vlk, R. Stern, D. Atzmon, R. Barták, Online multi-agent pathfinding, in: Proceedings of AAAI Conference on Artificial Intelligence (AAAI), 2019.
[62] J. Li, A. Tinka, S. Kiesel, J.W. Durham, T.S. Kumar, S. Koenig, Lifelong multi-agent path finding in large-scale warehouses, in: Proceedings of AAAI Conference on Artificial Intelligence (AAAI), 2021.
[63] H.W. Kuhn, The Hungarian method for the assignment problem, Nav. Res. Logist. Q. (1955).
[64] O. Gross, The bottleneck assignment problem, Tech. Rep., RAND Corp., Santa Monica, Calif., 1959.
[65] W. Hönig, T.S. Kumar, L. Cohen, H. Ma, H. Xu, N. Ayanian, S. Koenig, Multi-agent path finding with kinematic constraints, in: Proceedings of International Conference on Automated Planning and Scheduling (ICAPS), 2016.
[66] E. Boyrasky, A. Felner, G. Sharon, R. Stern, Don't split, try to work it out: bypassing conflicts in multi-agent pathfinding, in: Proceedings of International Conference on Automated Planning and Scheduling (ICAPS), 2015.
[67] E. Boyarski, A. Felner, R. Stern, G. Sharon, D. Tolpin, O. Betzalel, E. Shimony, Icbs: improved conflict-based search algorithm for multi-agent pathfinding, in: Proceedings of International Joint Conference on Artificial Intelligence (IJCAI), 2015.
[68] A. Felner, J. Li, E. Boyarski, H. Ma, L. Cohen, T.S. Kumar, S. Koenig, Adding heuristics to conflict-based search for multi-agent path finding, in: Proceedings of International Conference on Automated Planning and Scheduling (ICAPS), 2018.
[69] J. Li, A. Felner, E. Boyarski, H. Ma, S. Koenig, Improved heuristics for multi-agent path finding with conflict-based search, in: Proceedings of International Joint Conference on Artificial Intelligence (IJCAI), 2019.
[70] J. Li, D. Harabor, P.J. Stuckey, H. Ma, S. Koenig, Disjoint splitting for multi-agent path finding with conflict-based search, in: Proceedings of International Conference on Automated Planning and Scheduling (ICAPS), 2019.
[71] H. Zhang, J. Li, P. Surynek, S. Koenig, T.S. Kumar, Multi-agent path finding with mutex propagation, in: Proceedings of International Conference on Automated Planning and Scheduling (ICAPS), 2020.
[72] R.W. Floyd, Algorithm 97: shortest path, Commun. ACM (1962).
[73] K. Okumura, Y. Tamura, X. Défago, Time-independent planning for multiple moving agents, in: Proceedings of AAAI Conference on Artificial Intelligence (AAAI), 2021.